

Word embedding for semantic SQL in RDBMS

Lasse Wriedt

LASSE.WRIEDT@UNI-ULM.DE

Abstract

Relational databases dominate the storage of structured data, but natively cannot map latent semantic relationships between attributes. Bordawekar and Shmueli propose using word embeddings with ‘Semantic SQL’ to extend relational databases with semantic queries. While this approach was designed for large amounts of data, it is unclear how it behaves on small, highly structured datasets. This work therefore investigates, across five research questions, how parameter selection, similarity operators, and feature weighting affect embedding quality and semantic query results on a bank customer churn dataset. Based on distributional semantics, the Semantic SQL approach is applied to a bank customer dataset. In a Jupyter Notebook environment, CBOW and SGNS models are systematically trained with different parameters and evaluated using a specially developed quality metric (SSMQ). The results show that CBOW is superior to the SGNS algorithm on small datasets, while SGNS generates fragmented vector spaces. Different similarity operators capture different similarity concepts – from global profile similarity to partial matches. Feature weighting increases selectivity but does not improve the prediction of customer churn on its own. The work confirms the basic feasibility of Semantic SQL on small datasets, but also highlights the need for further research. This includes a systematic comparison with larger datasets and an investigation of the more advanced Semantic SQL concepts.

1. Introduction

As early as 2008, an analysis of web tables showed that a significant proportion of structured information on the web is available in tabular form and has relational structures that are fundamentally suitable for machine processing (Cafarella et al., 2008). This observation is also reflected in current market analyses: Relational databases continue to be the dominant standard for storing structured data (“DB-Engines Ranking”, 2025). Relational database management systems process billions of transactions worldwide every day and play a central role in economic, governmental, and financial application scenarios (“Relational Database Management System Market Report”, 2024).

Despite their structural strictness, these data collections often contain latent semantic relationships between attributes and data records that cannot be explicitly captured with classic SQL queries. Previous work addressed this shortcoming by annotating class labels for columns and relation labels for column pairs, especially in tables without headers or with meaningless attribute names. Among other things, this semantic enrichment improves applications such as table search and enables advanced operations such as joins or unions (Venetis et al., 2011).

Building on these approaches, Bordawekar and Shmueli show in their recent research how relational databases can be enhanced with semantic capabilities through so-called cognitive intelligence queries (CI queries). These CI queries are based on word embeddings, which derive latent syntactic and semantic relationships from row and column structures in relational databases (Bordawekar & Shmueli, 2016). The authors integrate this concept

into existing SQL query infrastructure to enable complex semantic queries (Bordawekar & Shmueli, 2017), and ultimately investigate advanced applications such as the analysis of disclosure levels (Bordawekar & Shmueli, 2019). However, their work leaves open how this approach performs on small, highly structured relational datasets, and which configurations and similarity operations yield meaningful results in practice. This work addresses these open questions by implementing and experimentally evaluating the Semantic SQL approach proposed by Bordawekar and Shmueli on a bank customer churn dataset (Topre, 2022). The theoretical analysis is complemented by a practical implementation in a Jupyter Notebook environment, in which all experiments are conducted and made reproducible. The following research questions guide the investigation:

- RQ1** How do Word2Vec training parameters influence the quality of the learned embeddings, and which parameter configurations yield optimal results for the chosen structured relational database?
- RQ2** To what extent are key vectors positioned in proximity to the attribute values of their corresponding database rows in the learned vector space?
- RQ3** How do different semantic proximity operators affect the resulting similarity scores and the structure of retrieved neighborhoods in relational embedding spaces?
- RQ4** What effect does attribute-level feature weighting have on semantic similarity rankings and increased discriminatory span?
- RQ5** To what extent can semantic similarity to a churn customer centroid be used to identify customers at risk of churning across different weighting configurations?

The remainder of this work is structured as follows. Section 2 introduces the theoretical foundations, covering the relational model, distributional semantics, the Word2Vec architecture, and the Semantic SQL framework proposed by Bordawekar and Shmueli. Section 3 describes the methodology, and Section 4 details the implementation. Section 5 presents the experimental results, which are subsequently discussed in Section 6. Section 7 concludes the work.

2. Literature Review and Background

This section introduces the formal foundations and strengths of the relational database model in order to analyze its central semantic weaknesses in mapping meaning and similarity. This distinction serves as a starting point for later justifying the necessity and potential of word embedding approaches to overcome these limitations and lastly introducing the work of Bordawekar and Shmueli (2017, 2019) that aims to integrate these approaches in Relational database management systems (RDBMS).

2.1 Relational databases and their semantic limitations

RDBMS are founded on the relational model introduced by Edgar F. Codd, in which data is represented as relations with fixed schemas and manipulated using relational algebra.

Relationships between entities are expressed implicitly via foreign keys, and queries are formulated declaratively using SQL. This model ensures high data consistency and reliability, particularly through transaction concepts formalized by the ACID properties (Codd, 1970; Gray & Reuter, 1993).

Despite their maturity and robustness, RDBMS exhibit fundamental limitations with respect to semantic expressiveness. Queries are based on deterministic predicate logic and operate primarily on exact matches or strict orderings. As a result, notions such as semantic similarity, relevance, or latent relationships are not natively supported. These limitations arise because relational databases do not model implicit correlations or latent semantic structures within the data. While numerical distances can be computed, comparisons based on meaning—such as similarity between texts, user interests, or behavioral patterns—require complex, application-specific logic outside the database system. Consequently, large volumes of semantically rich information stored in relational tables remain inaccessible to classical SQL-based querying.

To address these shortcomings, alternative database paradigms such as graph and semantic databases have been developed, which treat relationships as first-class entities and enable meaning-oriented queries. However, due to their established ecosystem and transactional guarantees, RDBMS remain dominant in practice, motivating research efforts that seek to integrate semantic and machine learning techniques directly into relational systems.

2.2 From symbolic models to distributional semantics

RDBMS share fundamental limitations with early approaches to natural language processing (NLP). Both rely on symbolic representations that can only be linked to explicitly predefined concepts, making it impossible to derive implicit meanings automatically from data or context. In both domains, these limitations motivated a paradigm shift away from handcrafted symbols and rules toward data-driven, context-based models. This shift is captured by the concept of distributional semantics.

Distributional semantics originates in linguistics and was introduced by Harris (1954). Its core idea is commonly summarized by Firth (1957): “You shall judge a word by the company it keeps.” According to this principle, the meaning of a word is determined by the contexts in which it occurs, operationalized through word co-occurrences. Unlike symbolic models, meaning is not defined by fixed rules or ontologies but is encoded in distributional vectors representing contextual usage patterns. This approach therefore maps discrete linguistic symbols into a continuous, multidimensional vector space in which semantic similarity is represented by geometric proximity. Within this framework, semantic similarity can be distinguished into attributional similarity, where words share common properties (e.g., “dog” and “cat”), and topical similarity, where words appear in similar thematic contexts (Landauer & Dumais, 1997; Turney, 2006). Proximity can be quantified either by vector distance or by directional similarity, the latter being more informative in high-dimensional spaces. Directional similarity is typically measured using cosine similarity, which captures the angle between two vectors independently of their magnitudes.

Distributional semantics is inherently scalable, as it is based on large co-occurrence statistics that can be processed efficiently using dimensionality reduction techniques. It is also largely domain-independent, since it does not rely on manually defined linguistic

resources (Ferrone & Zanzotto, 2020). Early vector space models relied on high-dimensional, sparse representations derived from raw co-occurrence counts. Modern approaches instead use low-dimensional, dense vector representations known as embeddings. These embeddings are learned rather than explicitly counted and are typically generated using neural methods such as word2vec, which efficiently operationalize the distributional hypothesis (Jurafsky & Martin, 2026).

2.3 Word embeddings and the Word2Vec model

Word2Vec refers to a family of efficient log-linear models for generating continuous word vectors from large text corpora (Mikolov, Chen, et al., 2013). The starting point is the distributional assumption that the meaning of a word can be approximated by its context. During training, words are represented based on their local environments in such a way that semantically and syntactically similar words receive similar vectors.

Mikolov, Chen, et al. (2013) propose two model variants: Continuous Bag-of-Words (CBOW) and Skip-Gram (SG). CBOW predicts a target word based on its context words, while Skip-Gram conversely predicts context words based on the target word. Both variants dispense with complex nonlinear structures, thereby enabling efficient scaling to very large corpora. In practice, CBOW shows advantages for frequent words, while Skip-Gram provides more robust representations for rare words.

Negative sampling is often used to reduce the computational effort (Mikolov, Sutskever, et al., 2013). This reduces the training problem to distinguishing observed word pairs from randomly generated negative examples. This approximation allows for efficient modeling even with very large vocabularies. The combination of skip-gram and negative sampling (SGNS) is now considered the standard method for static word vectors.

Theoretically, SGNS can be interpreted as implicit factorization of a word-context matrix (Levy & Goldberg, 2014). This creates geometric structures in which semantic relations can be mapped as vector operations. At the same time, classic Word2Vec models remain context-independent, as each word is assigned exactly one vector. Polysemy is thus insufficiently captured, and the representations reflect only statistical properties of the underlying corpus.

Practical implementations of Word2Vec are available through specialized libraries such as Gensim (Řehůřek, Radim et al., 2025), which provides optimized implementations of both CBOW and Skip-Gram with support for negative sampling and incremental training. This work utilizes Gensim’s Word2Vec implementation for all model training experiments. Furthermore, Word2Vec training involves several hyperparameters that significantly influence the resulting embeddings. Table 6 (Appendix A) summarizes the key parameters and their effects on model quality. The interaction between these parameters is complex and domain-dependent, motivating a systematic parameter exploration.]

2.4 Cognitive Intelligence Queries and vector operations in SQL

Relational database management systems are traditionally designed around exact, value-based predicates and therefore offer only limited support for capturing latent semantic relationships between data values. Word embedding models have shown that semantic meaning can be represented as a geometric structure in continuous vector spaces when

discrete symbols occur in comparable contexts. Although relational databases do not exhibit an inherent sequential structure, individual attribute values can only be interpreted meaningfully through their co-occurrence with other values within the same tuple. This observation motivates the transfer of vectorization and similarity concepts from natural language processing to structured relational data.

The concept of a *Cognitive Database* is introduced and systematically developed by Bordawekar and Shmueli (2016) and subsequently refined in Bordawekar and Shmueli (2017). A cognitive database extends a relational database system with semantic capabilities based on word embedding techniques. The central idea is to transform structured relational data into a text-like representation suitable for learning latent semantic relationships using NLP-based methods. While the 2016 work establishes the fundamental feasibility of this approach, the 2017 paper provides a detailed methodological and architectural elaboration. A distinct extension of this line of work is presented in Bordawekar and Shmueli (2019), which shifts the focus towards controlled information disclosure enabled by semantic representations.

As a foundational step, Bordawekar and Shmueli (2016, 2017) introduce the concept of textification. In this process, individual database rows are transformed into linear sequences of tokens derived from the attribute values of a tuple. Each row is processed independently and forms a single training context. All tokens within a row jointly define a uniform semantic context for one another, such that semantic meaning arises from co-occurrence rather than from individual attribute values. By analogy to natural language, a database row corresponds to a sentence and attribute values correspond to words. The resulting text corpus can be constructed from the entire database, from individual tables, or from relational views. The use of views allows the learning process to be focused on relevant subsets of the schema while excluding irrelevant attributes. Tokenization is performed across heterogeneous data types to obtain a uniform symbolic representation. Textual attributes are decomposed into word tokens, categorical values such as department names or status codes are included directly, numerical values may be mapped to predefined ranges or grouped using algorithmic clustering techniques, with the resulting cluster labels used as tokens. Missing values are represented explicitly using NULL tokens, ensuring that their systematic occurrence can be captured during training. In contrast to assumptions inherited by Word2Vec from natural language, relational data does not exhibit a semantically meaningful word order. Consequently, the linear ordering of tokens is purely technical and carries no semantic significance. All tokens within a row are weighted equally, and a cyclic neighborhood is assumed in which the first and last tokens are considered neighbors.

In Bordawekar and Shmueli (2017), primary and foreign keys are treated as central mechanisms for context expansion. Primary keys are considered neighbors of all tokens in their corresponding row in order to prevent loss of semantic information due to low token frequency. Foreign keys extend the context of a row by following references during textification. The referenced row is identified, tokenized, and its tokens are appended or integrated into the existing sequence. This mechanism results in an implicit semantic join that enables the modeling of latent relationships between entities that are only indirectly connected in the relational schema.

A further extension discussed primarily in Bordawekar and Shmueli (2017) and assumed in Bordawekar and Shmueli (2019) is the integration of external text corpora into the training process. Co-training with external sources such as Wikipedia enables the semantic

integration of concepts that do not explicitly occur in the database. Alternatively, externally pre-trained embedding models may be used directly for querying or as initialization for further training on database content. To ensure scalability, the system is designed to support incremental training, allowing new or modified rows to be incorporated as additional training sequences without requiring full retraining.

2.5 Semantic SQL and vector-based query processing

Building on the learned vector representations, Bordawekar and Shmueli (2017) propose the integration of semantic analysis directly into the existing SQL infrastructure without introducing a new query language. Semantic functionality is realized as an extension of relational query processing through user-defined functions (UDFs). These UDFs encapsulate vector-based operations, including the extraction of relevant tokens, retrieval of corresponding vectors from system tables, and computation of semantic measures such as similarity or analogy. As a result, semantic operations can be embedded transparently in `SELECT`, `WHERE`, `ORDER BY`, and `HAVING` clauses. For many operations, entire database rows are represented as vectors by aggregating the vectors of their constituent tokens, typically using centroid-based averaging. This enables semantic comparison at the tuple level and allows queries such as the retrieval of semantically similar customers or transactions.

These UDFs form the basis of so-called Cognitive Intelligence (CI) queries, which perform contextual rather than value-based comparisons. CI queries support similarity search, inductive reasoning, semantic clustering, and predictive queries involving tokens that do not explicitly appear in the database. The cosine similarity between vectors serves as the central similarity measure. By capturing directional similarity in high-dimensional embedding spaces, cosine similarity provides a robust basis for semantic comparison that is independent of absolute vector magnitude. Semantic similarity queries are conceptually interpreted as k -nearest neighbor searches in the embedding space. A common usage pattern is the retrieval of the top- k most similar tuples, implemented using `ORDER BY` on the similarity score in combination with `LIMIT`.

Beyond similarity search, CI queries also support inductive reasoning. Analogy queries are based on linear vector operations of the form $A - B + C \approx D$, where D denotes the token whose vector is closest to the resulting combination. Semantic clustering exploits the geometric structure of the embedding space to identify groups of related tokens or tuples, either through explicit clustering algorithms or implicit similarity thresholds. The odd-man-out problem is treated as a special case of inductive reasoning, identifying the element whose vector deviates most strongly from the group centroid. A key capability emphasized in Bordawekar and Shmueli (2019) is the use of external concepts in SQL queries. Tokens originating from external corpora or pre-trained models can be used directly in CI queries, even if they do not occur in the database itself. Through their vector representations, these external tokens become semantically linked to database entities within a shared embedding space.

In contrast to classical SQL, which relies on exact comparisons and Boolean predicates, CI queries operate on continuous semantic spaces. Instead of binary truth values, they produce numerical similarity scores that quantify relevance or semantic proximity. This

enables soft, context-dependent filtering and ranking, thereby extending SQL beyond its traditional capabilities.

A distinct focus of Bordawekar and Shmueli (2019) is the analysis of information disclosure enabled by semantic representations. The authors show that vector embeddings may implicitly encode sensitive information even when the underlying data is partially or fully encrypted. CI queries can therefore operate on encrypted tokens while preserving semantic relationships, leading to different degrees of information disclosure. This allows fine-grained control over which aspects of the data are revealed, but also introduces new security considerations. In particular, the use of external corpora may amplify unintended information leakage by linking database tokens to external semantic concepts.

3. Methodology

This section operationalizes the five research questions formulated in Section 1. The methodological design follows an experimental framework in which the Semantic SQL approach is implemented and systematically evaluated on a small, highly structured relational dataset.

RQ1 is addressed through a controlled parameter study of Word2Vec configurations and the definition of a composite embedding quality metric. RQ2 is operationalized by analyzing whether primary key embeddings are positioned in close proximity to the attribute values of their corresponding rows. RQ3 investigates the structural and statistical behavior of different semantic proximity operators. RQ4 examines the effect of attribute-level feature weighting on similarity rankings and discriminatory span. Finally, RQ5 evaluates whether similarity to a churn centroid can serve as an indicator for identifying customers at risk of churning.

The following subsections describe the experimental pipeline that enables the systematic investigation of these questions.

3.1 Data preparation

The purpose of data preparation is to transform structured relational data into a textual corpus suitable for distributional learning. Each table row is mapped to a single sentence, while the attribute values contained in the row are represented as tokens. This representation serves as the input for subsequent Word2Vec training. To ensure a uniform textual representation across heterogeneous SQL data types, attribute values are transformed according to the following principles.

Textual attributes (VARCHAR): Textual values may be normalized prior to tokenization in order to ensure consistent representation of semantically identical content. Multi-word expressions can be merged into composite tokens. Optionally, column names may be prefixed to tokens to provide additional contextual information.

Numeric attributes (NUMERIC, DECIMAL): Numeric values are converted into textual representations, either by direct string encoding or by abstraction through categorization or clustering. This enables the embedding model to capture contextual similarities beyond exact numeric equality.

3.2 Model training with Word2Vec

This subsection directly addresses RQ1, which investigates how Word2Vec training parameters influence the quality of learned embeddings in a structured relational context. Based on the generated text corpus, Word2Vec models are trained to learn semantic relationships between tokens derived from relational data. In contrast to natural language corpora, relational table rows do not exhibit an inherent word order. Consequently, each row is treated as an unordered set of tokens, and a bag-of-words representation is adopted in which all tokens contribute equally to the context. To achieve this, the words in the sentences are repeated multiple times. In addition, primary keys are treated as neighbors of all other tokens within the same row. This treatment is implemented by assigning each row a unique identifier token that is distributed multiple times in each sentence, so that every word is adjacent to it. As a result, the embedding of a primary key is expected to capture the semantic information of the corresponding table row, since it co-occurs with all other tokens from that row.

Word2Vec training involves several parameters that can significantly influence the resulting model. Due to the highly uniform structure of the generated corpus—characterized by limited size, fixed sentence lengths and consistent token counts—parameter selection is expected to have a pronounced effect on model behavior. An overview of the evaluated parameters is provided in Table 6 (Appendix A), based on the definitions in Mikolov, Chen, et al. (2013).

3.2.1 TEST SERIES

For the purpose of RQ1, optimal parameter configurations are defined with respect to the composite Semantic SQL Model Quality score (SSMQ), introduced in the following subsection. Parameter settings are compared quantitatively based on their resulting SSMQ values, enabling an objective selection of configurations that maximize embedding quality under the structural constraints of the dataset.

To systematically investigate the influence of training parameters, two complementary test series are conducted. The first series employs a one-at-a-time (OAT) analysis applied to the three central parameters—vector size, window size, and number of training epochs—as well as to the degree of word repetition in sentences. These parameters are varied across a wide range of values, including extreme settings, in order to observe their individual effects on model quality. The second test series performs a two-dimensional grid search to systematically evaluate combinations of vector size and window size, with remaining parameters fixed to standard values as specified in Table 4. Both test series include systematic comparison between the CBOW and SGNS architectures across all parameter configurations, enabling a comprehensive assessment of their relative performance on the relational corpus.

3.2.2 QUALITY ANALYSIS

Since Word2Vec is an unsupervised learning method, there is no explicit ground truth against which prediction errors can be directly measured. Consequently, classical supervised evaluation metrics are not applicable. For the purpose of RQ1, this work adopts a hybrid evaluation strategy that assesses model quality along three complementary dimensions: the geometric structure of the embedding space, reproducibility under repeated training,

and semantic plausibility with respect to domain-specific attributes. These dimensions are operationalized by the silhouette coefficient (SC), rank stability (RS), and feature coherence (FC), respectively.

The silhouette coefficient (SC) evaluates the geometric clustering quality of the learned embedding space by relating intra-class cohesion to inter-class separation, following the formulation proposed by Rousseeuw (1987). SC is computed over predefined semantic categories that group embedding instances according to their corresponding database attributes. Distances between embeddings are measured using cosine distance. To ensure compatibility with the aggregated model quality score, only non-negative silhouette values are considered. The formal definition of the global silhouette coefficient is provided in Appendix C.1. While SC captures geometric separability, it does not provide information about the robustness of the learned embedding structure. This aspect is addressed by the rank stability (RS) metric, which evaluates the reproducibility of local neighborhoods across multiple independent training runs. RS measures how consistently the top- k nearest neighbors of an embedding instance are preserved under repeated training with different random initializations. Neighborhood overlap is quantified using Jaccard similarity and aggregated across instances and runs. The formal definition of RS is given in Appendix C.2. To complement the intrinsic evaluation provided by SC and RS, feature coherence (FC) is employed as a semi-extrinsic plausibility measure. FC evaluates whether local neighborhoods in the embedding space align with domain-level characteristics that are intentionally excluded from the training process. For each embedding instance, FC measures the overlap between its derived feature labels and those of its nearest neighbors. Since the absolute magnitude of FC depends on marginal feature distributions, a random baseline is computed and used for normalization. Formal definitions of both the raw and normalized FC scores are provided in Appendix C.3.

For quantitative comparison and parameter optimization, the three metrics are combined into a single Semantic SQL Model Quality score (SSMQ). SSMQ is defined as a weighted linear combination of RS, SC, and FC, with weights controlling the relative influence of each metric. The formal definition of SSMQ is provided in Appendix C.4.

3.3 Proximity between keys and row value vectors

This subsection addresses RQ2, which tests the intuitive assumption that key embeddings that capture the semantic information of the row values are located more closely in the vector space to the value embeddings of their own row than to the values of different rows. Specifically, the analysis tests whether the k nearest neighbors of a key contain a disproportionately large number of tokens associated with its own row. Other key vectors are excluded from this analysis to avoid confounding effects due to key-key similarities.

Three evaluation metrics are employed for this assessment. The Average Proximity Score (APS) measures the proportion of row-specific values contained among the top- k neighbors of a key. The Weighted Average Proximity Score (WAPS) extends APS by assigning higher weight to values that appear closer to the key in the neighborhood ranking, thereby emphasizing stronger associations. Finally, the Model Score (MS) aggregates WAPS across multiple randomly selected keys to provide a model-level assessment of key embedding quality. Formal definitions of APS and WAPS are provided in Appendix C.5.

It is important to clarify that this test does not make any statement about the ability of key vectors to encode semantic information about their lines. This aspect is not considered in this work.

3.3.1 METHOD BENCHMARK OF SEMANTIC SQL SIMILARITY OPERATORS

This subsection addresses RQ3, which investigates how different semantic proximity operators affect similarity scores and the structural properties of retrieved neighborhoods in relational embedding spaces. Each operator induces a distinct aggregation mechanism over feature-level similarities and may therefore emphasize different aspects of relational proximity. A systematic comparison of these semantic similarity operators is conducted to assess their effectiveness in measuring relational proximity between database rows.

The analysis follows the proximity operators adapted from the lecture: PROXIMITYMAX, PROXIMITYAVG, PROXIMITYTOPNAVG, PROXIMITYAVGALL, and SUBSETPROXIMITYAVG. To compare sets of embedding vectors S_A and S_B , these five semantic similarity operators are employed, each implementing a distinct aggregation strategy over pairwise cosine similarities. PROXIMITYMAX computes the maximum cosine similarity over all vector pairs and captures the strongest local correspondence between the two sets (see Appendix B.1). PROXIMITYAVG measures the cosine similarity between the centroid vectors of S_A and S_B , thereby providing a global, row-level similarity estimate (see Appendix B.2). PROXIMITYTOPNAVG averages the top- N highest pairwise cosine similarities, emphasizing the most relevant matches while reducing the influence of weaker similarities. In the experiments, $N \in \{2, 5, 10\}$ (see Appendix B.3). PROXIMITYAVGALL computes the mean of all pairwise cosine similarities, assigning equal weight to every feature combination (see Appendix B.4). SUBSETPROXIMITYAVG applies PROXIMITYMAX to centroid vectors derived from all k -element subsets of S_A and S_B , capturing structured local similarities at the subset level. The parameter k is evaluated for $k \in \{3, 5, 7\}$ (see Appendix B.5).

The operators are evaluated along three dimensions. Statistical characterization analyzes the distribution of similarity scores across all reference-candidate pairs. Discriminatory power measures the separation between highly similar (top- k) and highly dissimilar (bottom- k) customers. Ranking consistency quantifies overlap between rankings produced by different operators using the Jaccard index. These criteria provide a comprehensive assessment of each operator’s ability to produce meaningful and distinguishable similarity judgments.

3.3.2 WEIGHT VECTOR CONTROL STUDY

This subsection first addresses RQ4 by analyzing the effect of attribute-level feature weighting on semantic similarity rankings and discriminatory span. The objective is to determine whether emphasizing selected attributes increases the separation between highly similar and dissimilar customers and alters ranking structures in a systematic manner.

While the proximity operators defined in Section 3.3.1 treat all feature embeddings equally, the lecture also considers weighted similarity computations using a feature-specific weight vector. This section evaluates the influence of such weighting schemes on semantic similarity assessments. Similarity between two rows is computed using a lecture-consistent weighted cosine formulation, in which feature-wise similarities are aggregated according to

a non-negative weight vector. The formal definition of the weighted similarity function is provided in Appendix C.6.

Several weight vector configurations are evaluated. A uniform baseline assigns equal weight to all features. In addition, the lecture’s predefined weighted configuration is used, along with two task-specific configurations that prioritize either demographic or financial attributes. Two experimental scenarios are considered to assess these configurations. In the first scenario, similarity rankings for selected reference customers are compared across weight configurations using ranking overlap curves. In the second scenario, a churn prototype is constructed from churned customers, and similarity scores to this prototype are used to generate rankings under different weight vectors.

The impact of feature weighting is evaluated in terms of discriminatory span, ranking overlap, and sensitivity to the choice of proximity operator. By jointly varying both similarity operators and weight vectors, the analysis assesses which combinations yield robust and discriminative semantic similarity results.

Building upon the weighting analysis of RQ4, RQ5 evaluates whether semantic similarity to a churn customer centroid can serve as an indicator for identifying customers at risk of churning. A centroid vector is constructed from embeddings of churned customers, and similarity scores between this centroid and all other customers are computed under different weighting configurations.

4. Implementation

The implementation uses a PostgreSQL database containing the Bank Customer Churn Dataset from Topre (2022). Analysis and model training are conducted in Jupyter Notebook (Project Jupyter, 2025) using a custom Python package that encapsulates the experimental workflow. All code and dependencies are containerized to ensure reproducibility. All code and configuration files required to reproduce the experiments are publicly available at: <https://github.com/QuiriniusvonBayern/DepOp-Seminar>.

4.1 Data Preparation

All database rows are loaded into the notebook environment. Each row is transformed into a textual representation following the procedure defined in Section 3.1.

Table 1: Selected unchanged customer data

customer_id	credit_score	gender	estimated_salary	churn
15634602	619	Female	101348.88	1
15647311	608	Female	112542.58	0
15619304	502	Female	113931.57	1

Automated clustering of large quantities of numerical values is central to the preparation process. In addition, all numerical values are converted into textual representations and the column name is appended to each label. The customer identification number has been removed because it is added arbitrarily and does not contain any semantic information. This results in the values in Table 2, for example.

Table 2: Selected customer data after the first step of preparation

credit_score	gender	estimated_salary	churn
Credit_Score_Fair	Gender_Female	Salary_High	Churn_Yes
Credit_Score_Fair	Gender_Female	Salary_High	Churn_No
Credit_Score_Poor	Gender_Female	Salary_High	Churn_Yes

In the final step of data preparation, keys are inserted that occur as neighbours for each row value. The result can be seen in Table 3. These keys replace the primary key, which did not include any semantic information in this database.

Table 3: Selected data after the second step of preparation

key1	credit_score	gender	key2	estimated_salary	churn	key3
key_1	Credit_Score_Fair	Gender_Female	key_1	Salary_High	Churn_Yes	key_1
key_2	Credit_Score_Fair	Gender_Female	key_2	Salary_High	Churn_No	key_2
key_3	Credit_Score_Poor	Gender_Female	key_3	Salary_High	Churn_Yes	key_3

The table is then converted into a sentence containing all row values as individual words. The values prepared in this way can then be used for training with Word2Vec. The churn column is excluded from training but retained in the dataset for validation purposes in later experiments.

4.2 Model Training with Word2Vec

The Word2Vec implementation from Gensim (Řehůřek, Radim et al., 2025) is used. OAT and grid search procedures are implemented as described in Section 3.2. For parameters not explicitly varied, default values from Appendix A are used. For each parameter configuration, six models are trained: CBOW and SGNS, each with three seeds (1, 42, 66).

OAT Test Series. Models are trained according to the parameter values specified in Table 4. For sentence length experiments, all tokens within a sentence are repeated according to the specified repetition factor.

Grid Search Test Series. A grid search is conducted using the parameter combinations listed in Table 5. All trained models are stored together with their parameter configurations.

The trained models are saved together with their model parameters.

4.3 Implementation of quality metrics

The quality metrics SC and FC do not work directly with word vectors, but rather with vectors of entire sentences. For this reason, sentence vectors are also created here. To do this, the vectors of the words associated with each database row are collected and used to calculate the average vector. The sentence vectors are also stored together with metadata.

When calculating the SC, the columns of the database table are used as semantic categories. For all words contained in a column, the vectors are then extracted from the model

Table 4: Parameter values for the OAT test series.

Parameter type	Number	Values
Sentence Length	5	1, 5, 20, 50, 100
Vector Sizes	18	20, 30, 40, 50, 60, 80, 100, 120, 140, 150, 160, 180, 225, 250, 275, 300, 350, 400
Window Sizes	19	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20
Epochs	23	3, 5, 7, 10, 13, 15, 17, 20, 23, 25, 28, 30, 35, 40, 45, 50, 60, 70, 80, 90, 100, 150, 200
To be trained: 65 parameters · 2 algorithms · 3 seeds = 390 models		

Table 5: Parameter values for the grid search test series.

Parameter type	Number	Values
Vector sizes	5	50, 100, 150, 200, 300
Window Sizes	6	2, 4, 6, 10, 15, 20
To train: 5 · 6 parameters · 2 algorithms · 3 seeds = 180 models		

and their average distance from each other is calculated. The average distance to the nearest foreign category is then calculated. Negative silhouette values are set to zero, as prescribed by the metric definition in formula C.1.

Rank stability (RS) is computed across the three seed variations of each model. For 100 randomly selected anchor tokens, the k -nearest neighbors are determined in each model variant using cosine similarity, with $k \in \{5, 10, 20\}$. Neighborhood sets are compared pairwise across seed variations using the Jaccard index. The RS score for each anchor is obtained by averaging across all k values and run pairs. Finally, anchor-level scores are averaged to yield the global RS metric.

Subsequently, feature coherence (FC) was implemented using a set of derived categorical descriptors that serve as validation features. These descriptors are deterministically constructed from selected database attributes and are explicitly excluded from the textification process, ensuring that they do not occur as training tokens in the Word2Vec models.

Each database row is associated with a small set of such derived labels, capturing high-level domain characteristics such as risk, customer value, and engagement. These labels form the feature set $F(r)$ of a reference instance r .

For each model and training seed, sentence vectors are extracted and the $k = 10$ nearest neighbors of each reference instance are determined using cosine similarity. Feature coherence is computed by calculating Jaccard similarity between the derived feature sets of each reference instance and its neighbors, averaged across all instances. A normalized FC score is obtained by comparing this value against a random baseline. The procedure is repeated for all three random seed initializations, and resulting FC values are averaged.

Finally, the three metrics are combined into the SSMQ score using the weighting scheme defined in Section C.4: $\alpha = 0.5$ for RS, $\beta = 0.3$ for SC, and $\gamma = 0.2$ for FC.

RS receives highest weight ($\alpha = 0.5$) as rank stability is a necessary condition for reliable semantic querying. SC is weighted with $\beta = 0.3$, reflecting the importance of geometric structure. FC receives lowest weight ($\gamma = 0.2$) as it provides only indirect validation through excluded features. These weights reflect the relative importance of reproducibility over geometric structure and domain plausibility.

4.4 Implementation of the key test

For each model, 50 keys per model are tested here. To implement the key vector quality measurement, the corresponding row values are first extracted from the DataFrame for each key to be evaluated. Since the direct neighborhood search would also return other key vectors, an extended search strategy is implemented: Initially, significantly more neighbors are retrieved (10000 by default), from which all key tokens are then filtered to obtain the desired 20 non-key neighbors.

For each of these neighbors, a check is performed to see whether it belongs to the model’s own row values. Found values are documented with their rank position and similarity, while missing values are recorded separately. The Average Proximity Score (APS) and Weighted Average Proximity Score (WAPS) are computed according to Equations C.5. The model score (MS) is obtained by averaging WAPS across all evaluated keys.

4.5 Implementation of the benchmark of Semantic SQL similarity operators

To ensure diverse semantic coverage, a set of five reference customers is selected using k -means clustering ($k = 5$) applied to the embedding-based feature representation of all customers. One representative customer is selected from each cluster by maximizing the distance to the corresponding cluster centroid, thereby sampling across distinct semantic regions of the embedding space.

For each reference customer and proximity operator, similarity scores are computed for all other customers, resulting in complete rankings. The operators are evaluated along three dimensions:

1. **Statistical characterization:** Distribution analysis of similarity scores
2. **Discriminatory power:** Separation between top-200 and bottom-200 customers
3. **Ranking consistency:** Jaccard overlap between operator-specific rankings

Parameter variants are evaluated for ProximityTopNAvg with $N \in \{2, 5, 10\}$ and for SubsetProximityAvg with $k \in \{3, 5, 7\}$.

4.6 Implementation of the Weight Vector Control Study

The weight vector control study implements the methodology from Section 3.3.2. Four weight configurations are evaluated: uniform ($w^{(\text{avg})}$), lecture-weighted ($w^{(\text{weighted})}$), demographic-focused ($w^{(\text{demo})}$), and financial-focused ($w^{(\text{fin})}$).

Two scenarios are implemented:

Scenario A (Similar customer identification): For each reference customer, weighted similarity scores are computed to all other customers using the lecture-consistent weighted formulation (Section C.6). Rankings are compared to the unweighted baseline using overlap curves.

Scenario B (Churn propensity estimation): A churn prototype is constructed by averaging feature embeddings of all churned customers. For each non-churned customer, churn propensity scores are computed as weighted cosine similarity to the prototype. Rankings under different weight vectors are compared using overlap analysis and Precision@K metrics.

5. Results

This section presents the experimental results. Visualizations are created using matplotlib (Matplotlib Development Team, 2025) and seaborn (Seaborn Developers, 2025). Dimensionality reduction to two dimensions is performed using t-SNE from scikit-learn (scikit-learn Developers, 2025).

5.1 Results of the OAT experiment

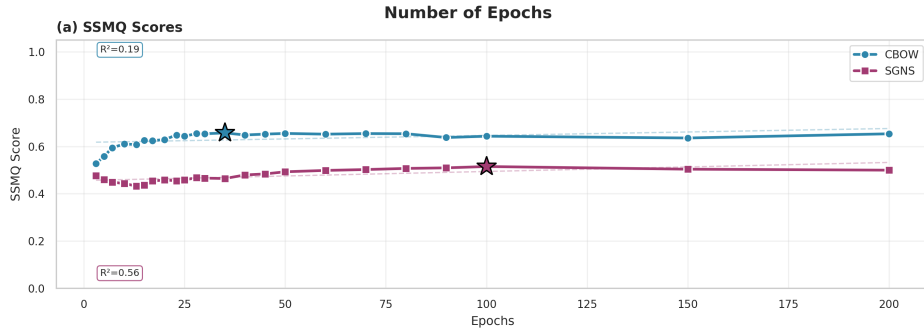


Figure 1: Impact of training epochs on model quality metrics. Top: SSMQ scores for CBOW and SGNS.

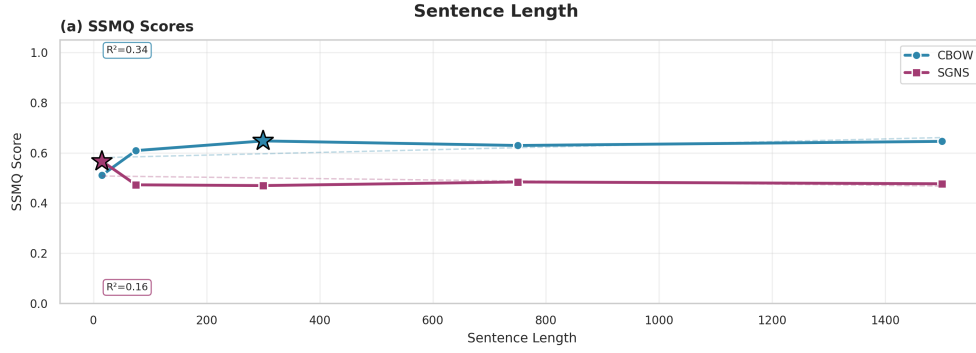


Figure 2: Impact of sentence length (word repetitions) on model quality metrics. Layout as in Figure 1.

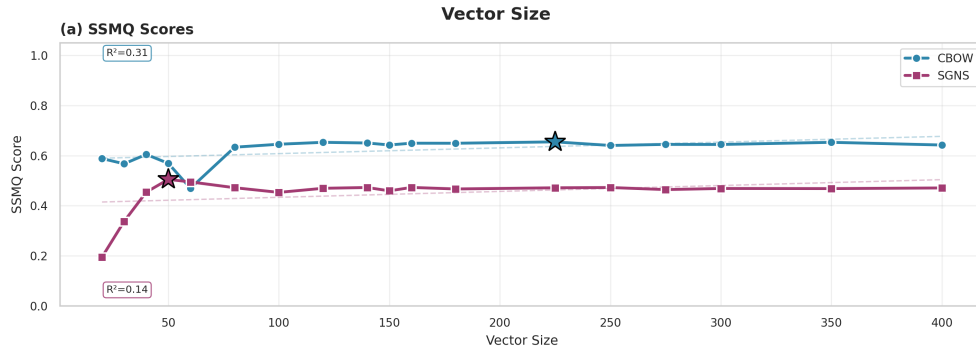


Figure 3: Impact of embedding vector size on model quality metrics. Layout as in Figure 1.

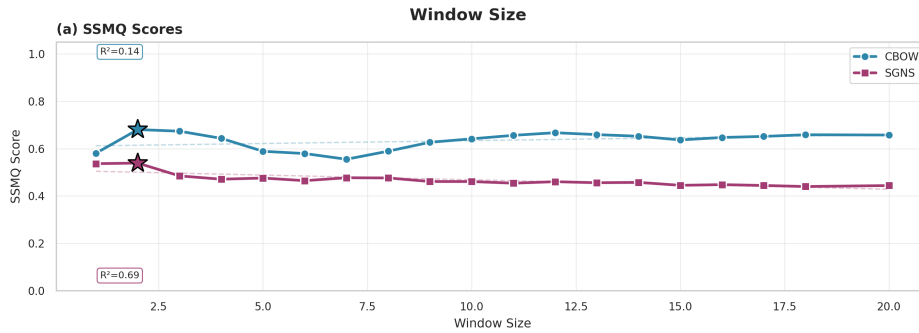


Figure 4: Impact of context window size on model quality metrics. Layout as in Figure 1.

Figures 1–4 show the results of the OAT analysis. Each figure displays the SSMQ scores for both algorithms, while the individual metric components (SC, FC, RS) separately for CBOW (left) and SGNS (right) can be found in Appendix D.

The results show that SGNS performs noticeably worse than CBOW, primarily due to the SC metric. SGNS achieves very low silhouette coefficients across most parameter

ranges (SC values consistently below 0.2), with slightly better performance only at low window sizes. For CBOW, SC values stabilize after initial parameter increases. FC and RS metrics show relatively stable performance for both algorithms. All parameters show the strongest influence on model quality at lower values, with quickly diminishing effects that result in a plateau at higher values. Extreme parameter settings do not yield notably better or worse performance. Optimal SSMQ scores appear relatively stable across a range of moderate parameter values, suggesting robust performance within a broad configuration space.

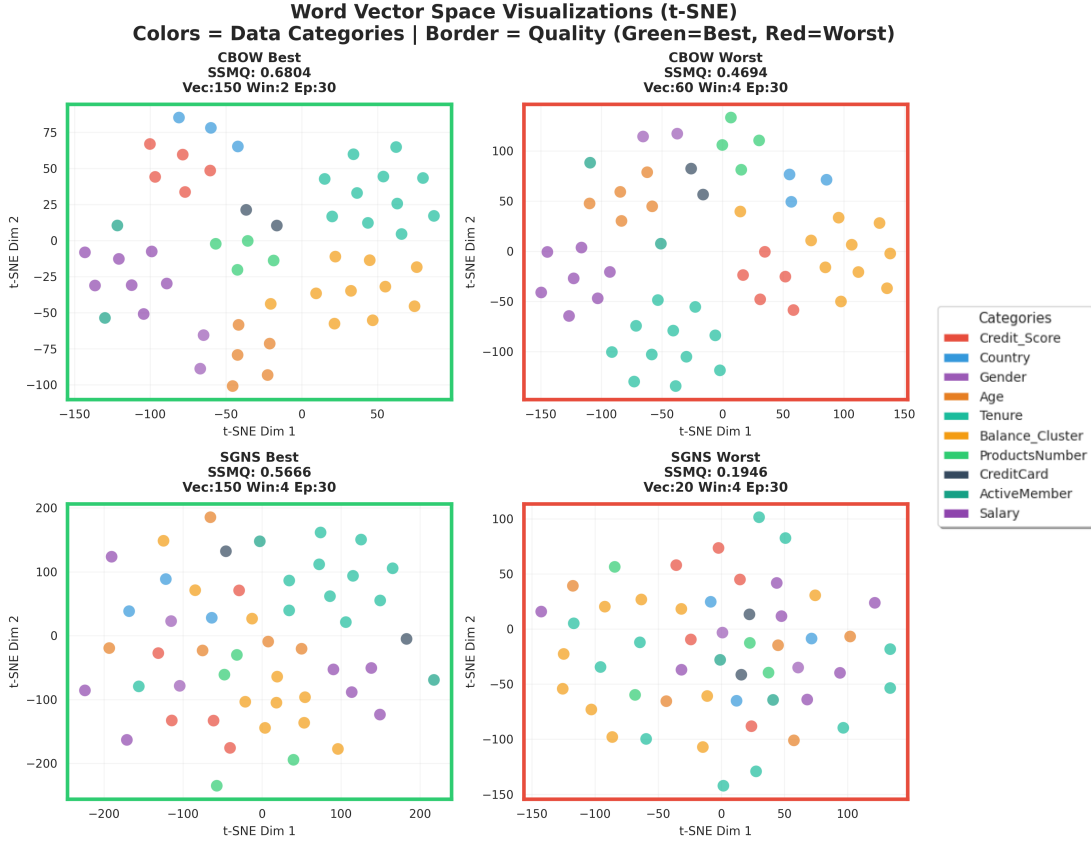


Figure 5: t-SNE visualization of word embeddings colored by database column. Top row: CBOW models (best left, worst right). Bottom row: SGNS models (best left, worst right). Model quality determined by SSMQ score.

Figures 5 and 6 visualize the embedding spaces of the best and worst performing models according to SSMQ. In Figure 5, word embeddings are colored by their source database column. CBOW exhibits clear clustering by column membership in both best and worst models, with well-separated semantic groups visible in the t-SNE projection. In the best CBOW model, distinct clusters are visible for all categories. The worst CBOW model shows more overlap but maintains recognizable cluster structure. In contrast, SGNS shows minimal clustering structure in both configurations. The best SGNS model displays heavily



Figure 6: t-SNE visualization of sentence embeddings colored by customer gender. Top row: CBOW models. Bottom row: SGNS models.

overlapping categories with no clear spatial separation. The worst SGNS model shows complete mixture of all categories across the embedding space.

Figure 6 shows sentence embeddings colored by customer gender. For CBOW, the best model exhibits stronger spatial separation between male (green) and female (red) customers, with distinct regional concentrations visible in multiple areas of the plot. The worst CBOW model shows more intermixing, though some gender-based clustering remains detectable. For SGNS, gender-based clustering remains weak in both cases. The best SGNS model shows only marginal cluster formation, with male and female points heavily intermixed. The worst SGNS model displays near-complete overlap between genders.

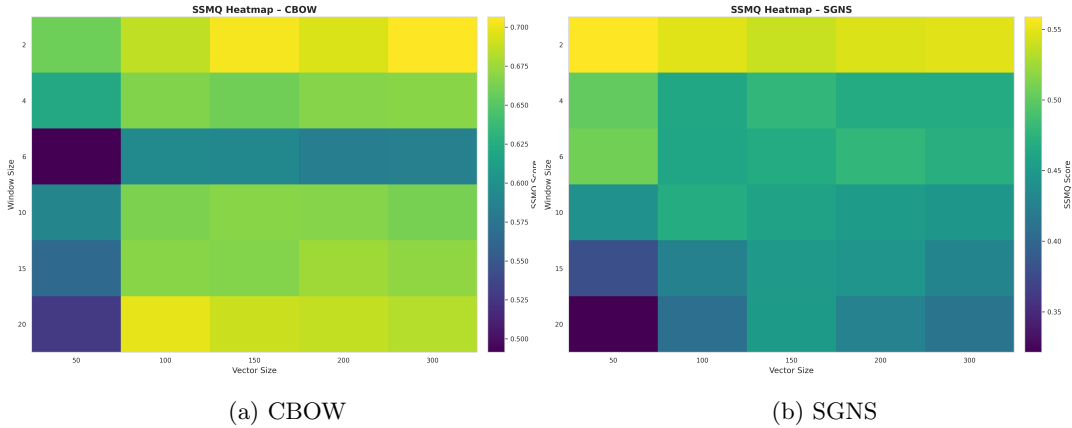


Figure 7: SSMQ scores across vector size and window size combinations (grid search). (a) CBOW shows robust performance across most configurations, with degradation only at very low vector sizes. (b) SGNS performs best at low window sizes and shows higher sensitivity to parameter changes.

5.2 Results of the grid search experiment

Figure 7a and 7b shows SSMQ scores as heatmaps (bright colors indicate high scores, dark colors low scores). The results confirm the OAT findings: both algorithms perform best at lower window sizes, but CBOW prefers higher vector sizes while SGNS performs better with lower dimensional embeddings. CBOW exhibits greater robustness to parameter variation, with degraded performance only at very low vector sizes.

5.3 Results of the key vector experiment

The key vector experiment shows minimal evidence that primary key embeddings are located in the vicinity of the associated row value embeddings. Figure 8 displays KVC model scores across all OAT parameters. Low R^2 values (ranging from 0.00 to 0.59) indicate weak correlation between row value vectors found close to the key vector and the tested parameter. The infrequent occurrence of row values among their keys suggests that observed matches are largely attributable to chance.

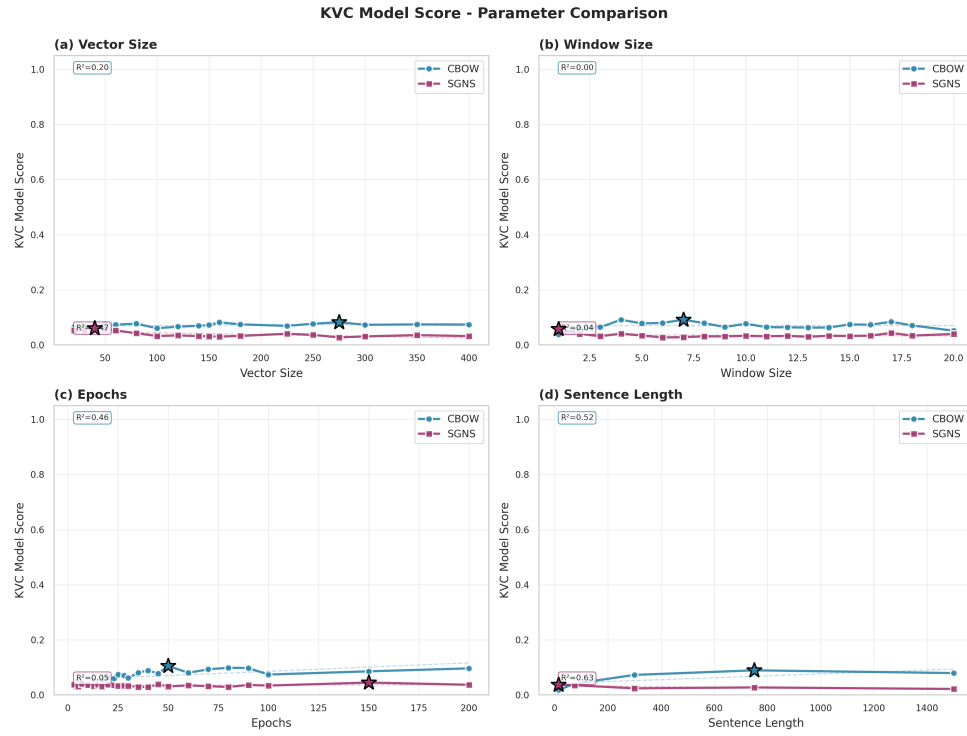


Figure 8: Key Vector Coherence (KVC) model scores across OAT parameters. (a) Vector size, (b) Window size, (c) Epochs, (d) Sentence length. Trend lines show weak correlation between parameters and key-row proximity (R^2 values indicate limited predictive power).

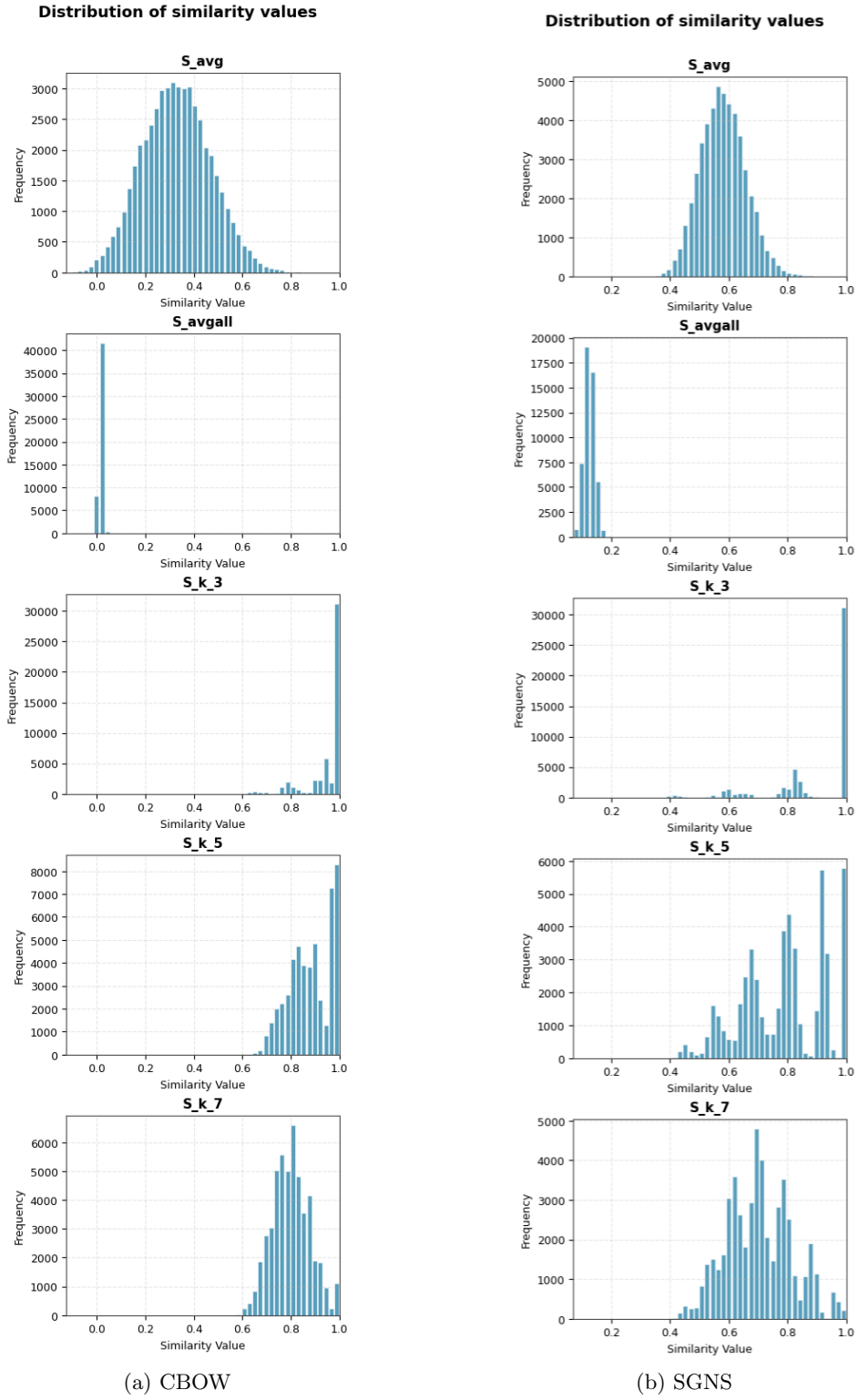


Figure 9: Similarity score distributions for proximity operators (Part 1). Each histogram represents approximately 50,000 reference-candidate pairs.

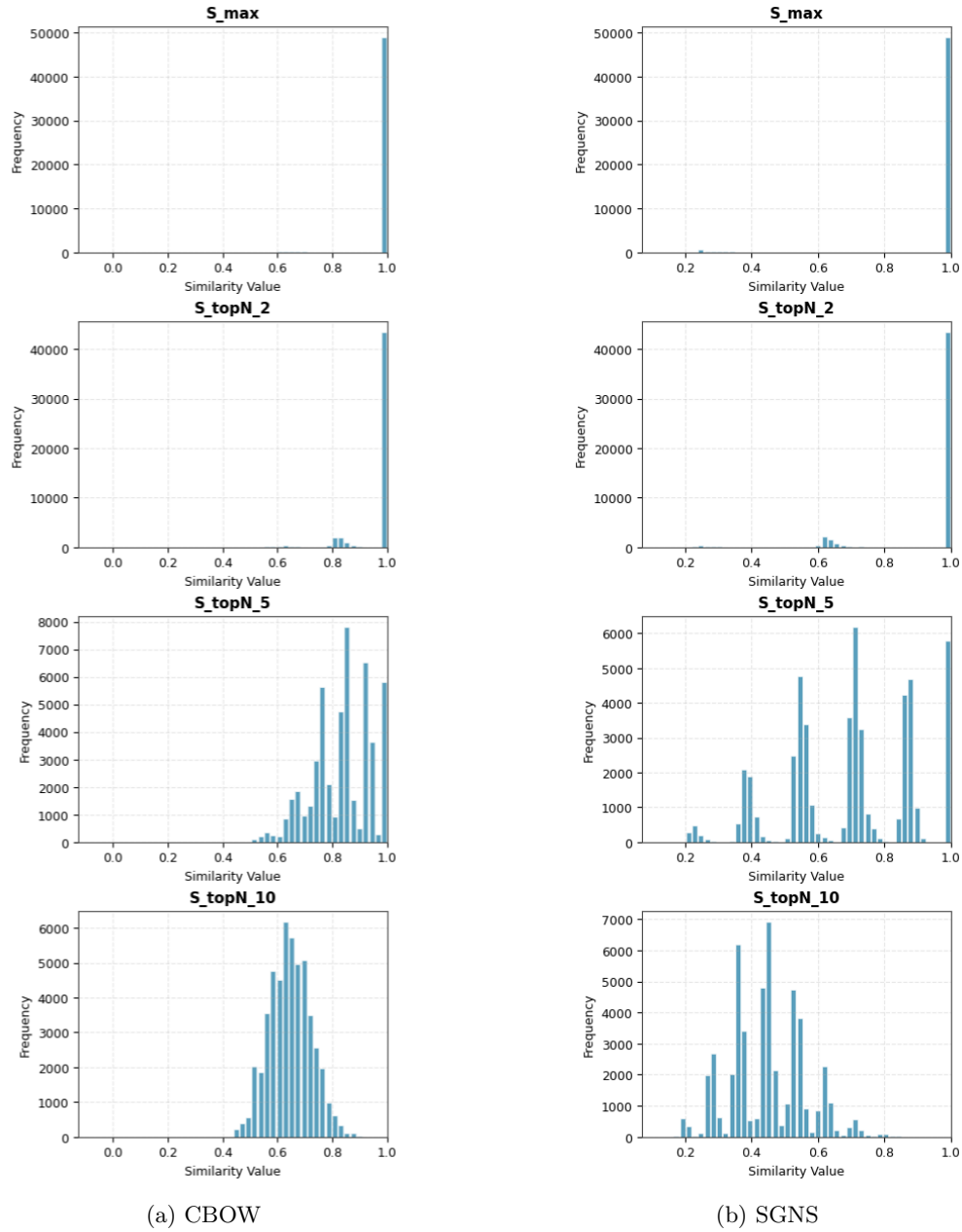


Figure 10: Similarity score distributions for proximity operators (Part 2). Layout as in Figure 9.

5.4 Results of the Semantic SQL Similarity Operator Benchmark

Figures 9 and 10 show similarity score distributions for CBOW (panels a) and SGNS (panels b). Each histogram represents approximately 50,000 reference-candidate pairs across five reference customers.

For CBOW, the centroid-based operator S_{avg} shows a broad, smooth distribution, indicating a global notion of similarity with moderately similar scores for many customers. S_{avgall} concentrates strongly around zero, suggesting that averaging all pairwise similarities eliminates semantic contrast. Partial similarity operators (S_{topN} , S_k) produce higher similarity values while preserving distributional spread. S_{max} exhibits strong accumulation near one, reflecting frequent high similarity from single strong attribute matches.

For SGNS, score distributions differ markedly from CBOW. S_{avg} shifts toward higher values with reduced variance, suggesting emphasis on local correlations. Operators S_{topN} , S_k , and S_{max} exhibit structured, often multimodal distributions.

Figure 11 shows the discriminatory power of proximity operators, measured as the mean similarity difference between top-200 and bottom-200 customers. For CBOW, S_{avg} achieves the highest separation, indicating suitability for global similarity ranking. S_{avgall} shows minimal separation and provides little discriminative value. For SGNS, the ranking changes markedly: S_{topN} variants achieve highest separation, followed by S_{max} and S_k . This demonstrates that operator effectiveness depends strongly on the embedding algorithm.

Figure 12 shows ranking consistency measured by Jaccard overlap. For both models, bottom-200 rankings exhibit high overlap across operators, indicating stable notions of dissimilarity. Top-200 rankings show lower overlap, particularly for CBOW, suggesting that different operators capture distinct types of semantic similarity.

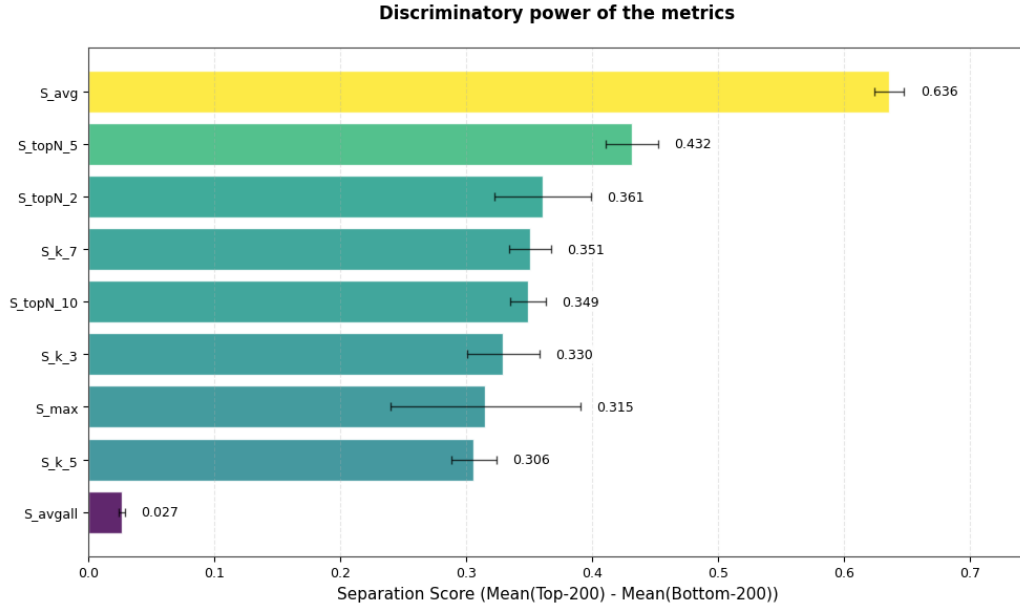
5.5 Results of the Weight Vector Control Study

5.5.1 SCENARIO A: SIMILAR CUSTOMER IDENTIFICATION

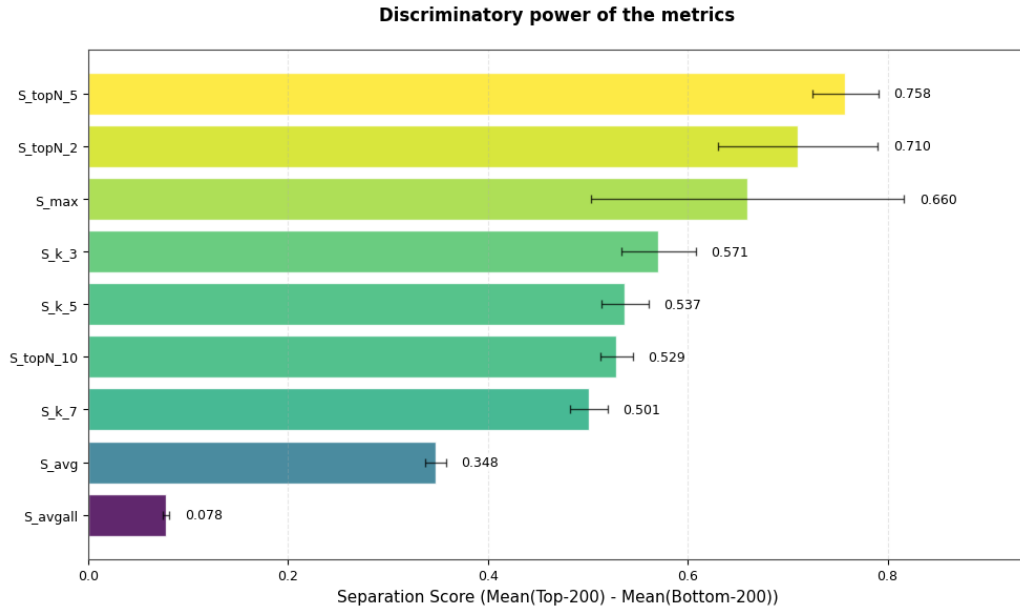
Figure 13 summarizes the discriminatory span of weighted similarity scores. For CBOW, similarity range and standard deviation remain stable across weight vectors, with negligible negative values. This indicates that CBOW produces compact, predominantly positive distributions that are only moderately affected by feature weighting.

In contrast, SGNS exhibits substantially higher sensitivity. Demographic weighting produces pronounced increases in both range and standard deviation, accompanied by notable proportions of negative similarity values. This suggests that feature-focused weighting amplifies semantic contrast in SGNS embeddings but introduces greater variability and polarity.

Figure 14 illustrates ranking stability under feature weighting. For both models, $w^{(\text{weighted})}$ shows highest overlap with the unweighted baseline, preserving much of the original ranking structure. In contrast, $w^{(\text{demo})}$ and $w^{(\text{fin})}$ produce substantially lower overlap, particularly for top-K rankings. This indicates that feature-specific weighting alters similarity notions and identifies markedly different sets of most similar customers. Across all weight vectors, bottom-K rankings show higher overlap than top-K rankings, indicating more stable notions of dissimilarity.



(a) CBOW



(b) SGNS

Figure 11: Discriminatory power of proximity operators measured as mean similarity difference between top-200 and bottom-200 customers. (a) CBOW: S_{avg} achieves highest separation. (b) SGNS: S_{topN} variants show strongest discrimination.

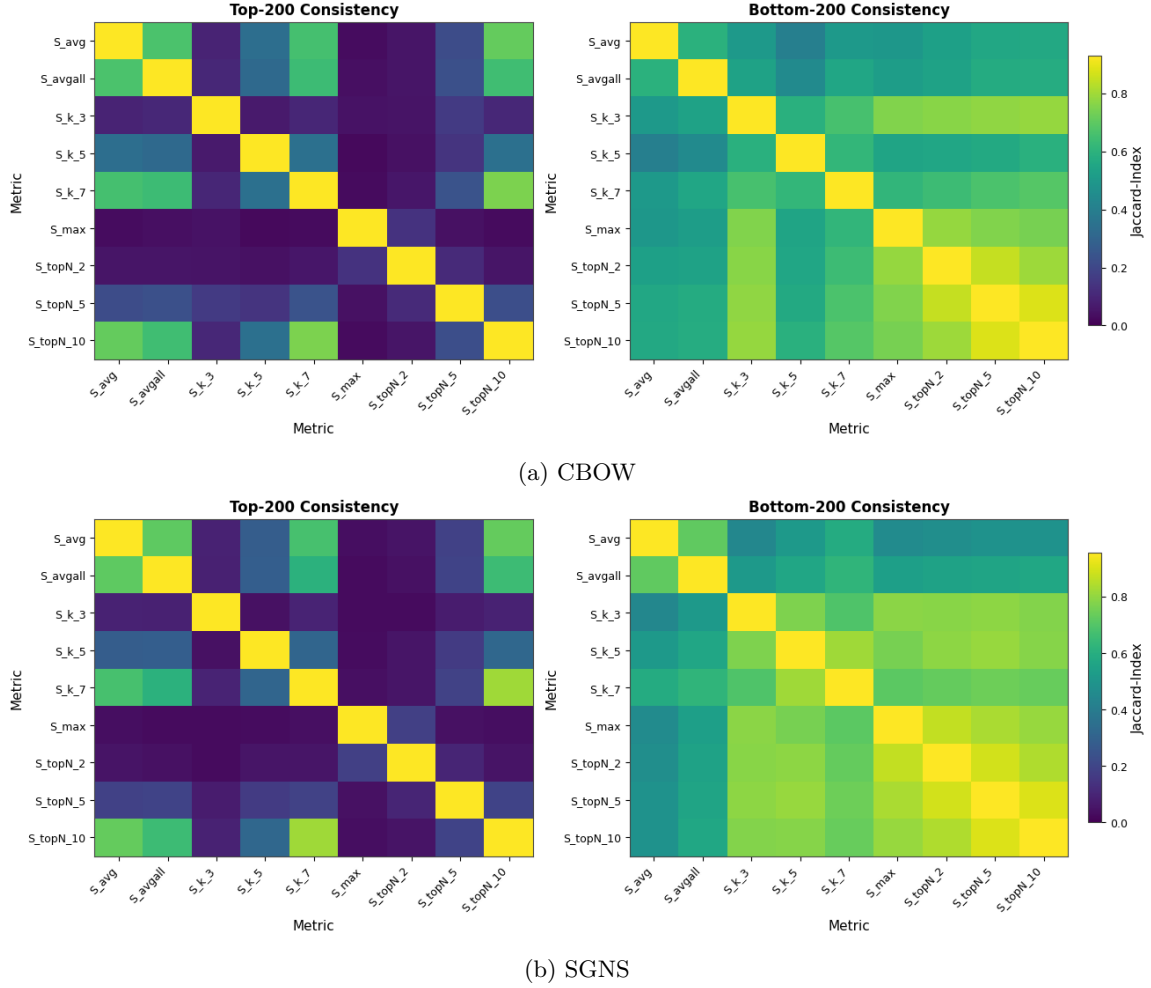


Figure 12: Ranking consistency between proximity operators measured by Jaccard overlap. Top panels: Top-200 customers. Bottom panels: Bottom-200 customers. (a) CBOW shows low Top-200 overlap, indicating distinct semantic neighborhoods. (b) SGNS exhibits higher overlap among S_{topN} and S_k operators.

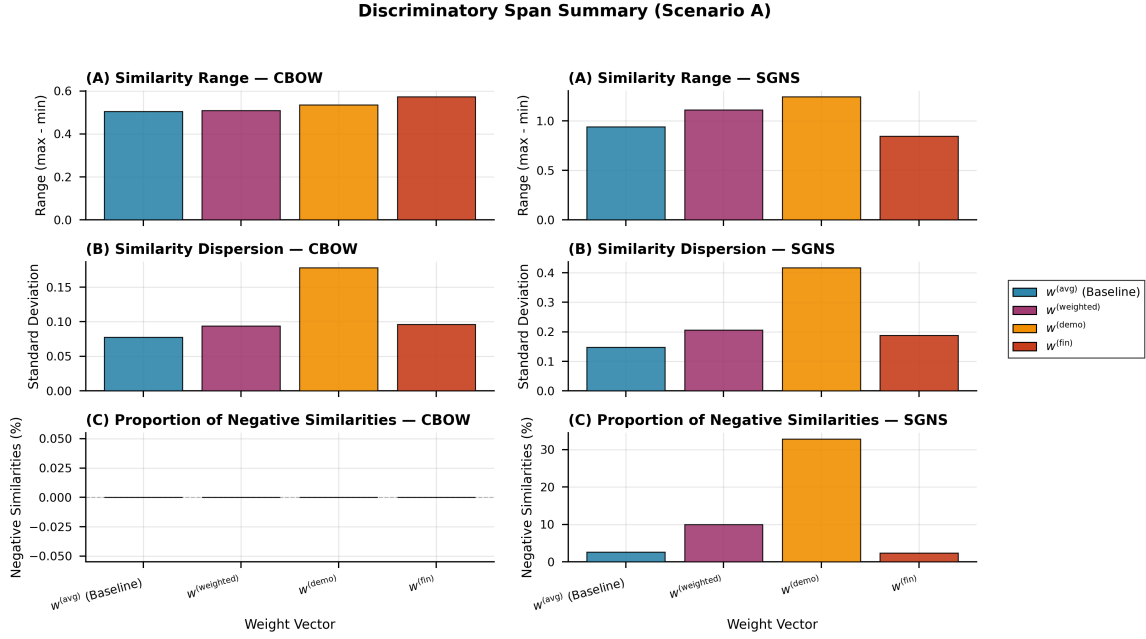


Figure 13: Discriminatory span of weighted similarity scores (Scenario A). Bars show similarity range (max-min), standard deviation, and proportion of negative values for each weight vector. CBOW exhibits stable distributions across weight configurations. SGNS shows increased variability and negative values under demographic weighting.

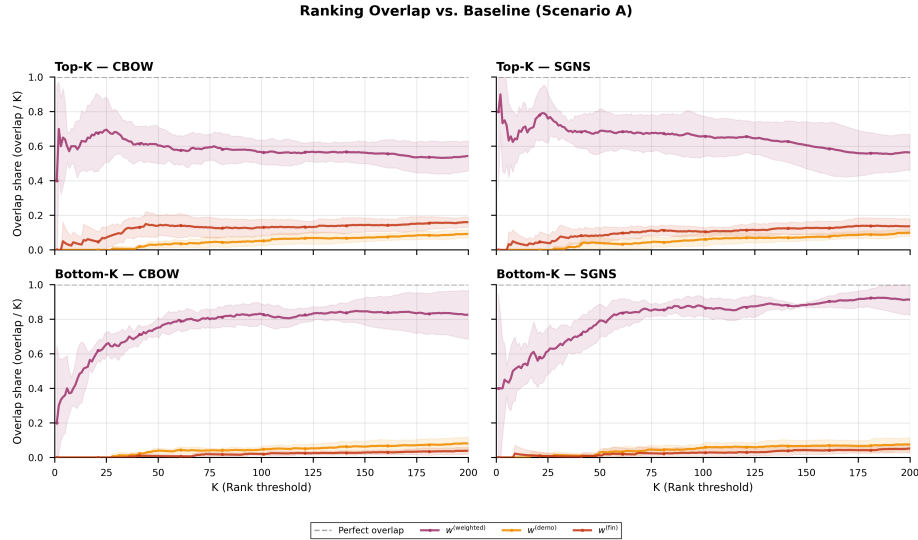


Figure 14: Ranking overlap with unweighted baseline (Scenario A). Curves show Top-K and Bottom-K overlap for different weight vectors. Lecture-weighted configuration ($w^{(weighted)}$) shows highest overlap. Demographic and financial weightings produce substantial ranking deviations, particularly in top-K positions.

5.5.2 SCENARIO B: CHURN PROPENSITY ESTIMATION

Scenario B evaluates how weight vectors affect churn propensity scoring using ground-truth churn labels.

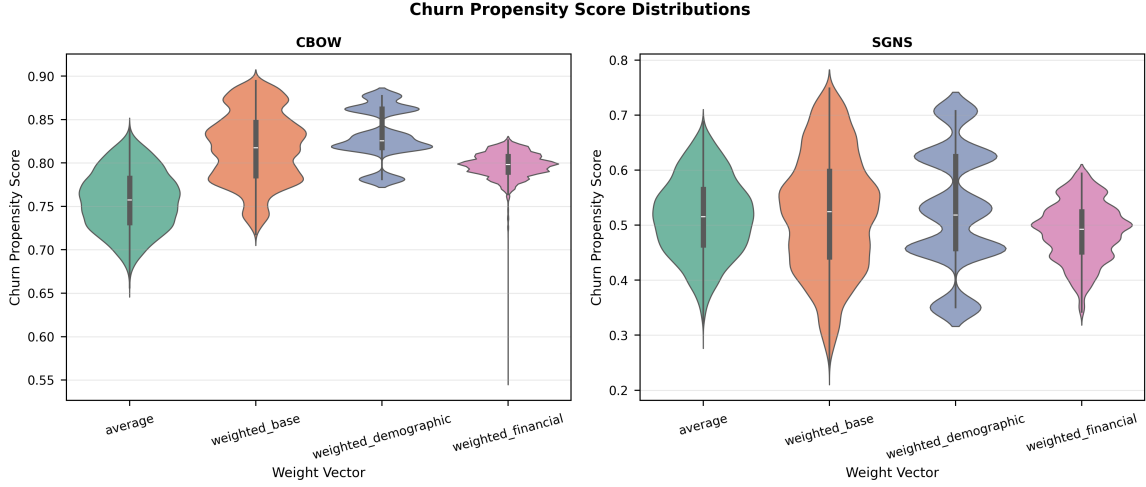


Figure 15: Churn propensity score distributions under different weight vectors (Scenario B). Violin plots for CBOW (left) and SGNS (right) show systematic shifts in location and dispersion. Unweighted and lecture-weighted configurations yield compact distributions; demographic and financial weightings produce broader, more heterogeneous distributions.

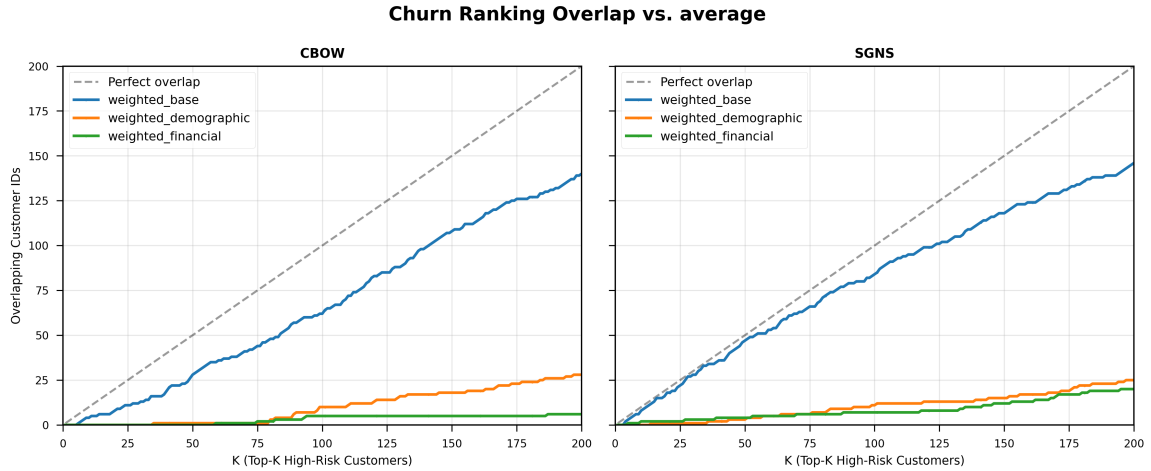


Figure 16: Churn ranking overlap with unweighted baseline (Scenario B). Curves show Top-K overlap for high-risk customer identification. Pattern similar to Scenario A: lecture-weighted configuration maintains highest overlap, while demographic and financial weightings produce substantial deviations.

Figure 15 shows churn propensity score distributions. For both models, feature weighting induces systematic shifts in location and dispersion. Unweighted and lecture-weighted configurations yield compact distributions, while demographic and financial weightings produce

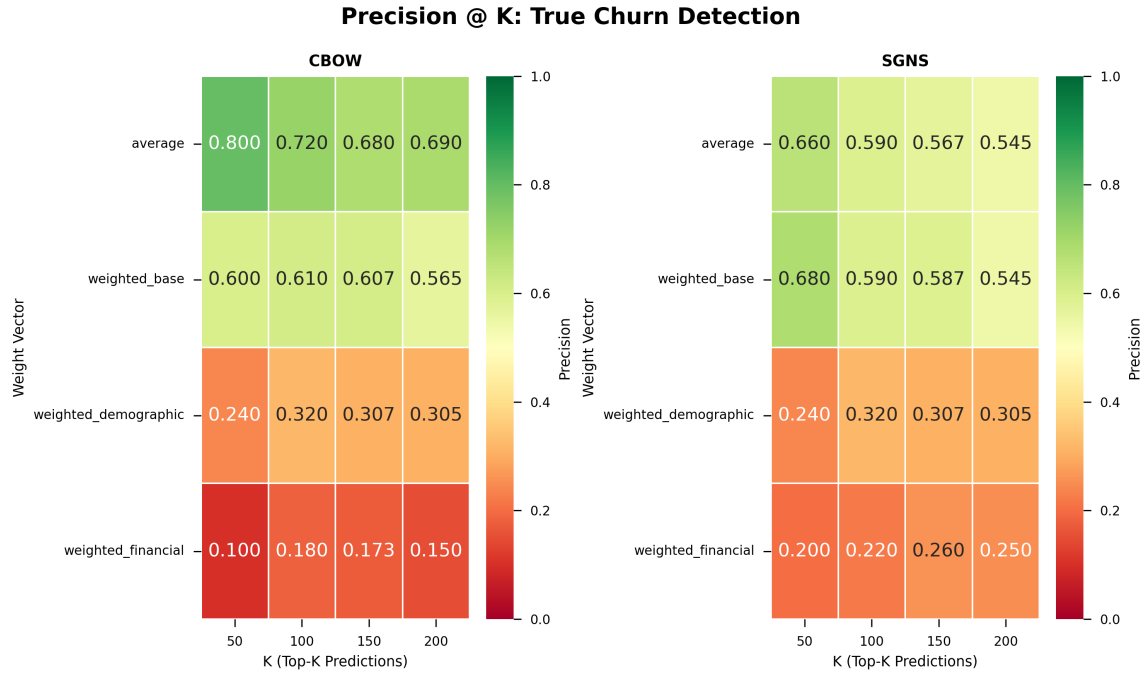


Figure 17: Precision@K for churn detection (Scenario B). Heatmaps show fraction of true churned customers among top-K predictions. Unweighted baseline achieves highest precision for both models. Feature-focused weighting (demographic, financial) degrades detection performance despite increasing discriminatory span.

broader, more heterogeneous distributions. Absolute score ranges differ between CBOW and SGNS, reflecting model-specific scaling effects.

Figure 16 shows churn ranking stability. Similar to Scenario A, $w^{(\text{weighted})}$ remains closest to baseline, while $w^{(\text{demo})}$ and $w^{(\text{fin})}$ produce substantial deviations in top-K high-risk sets. Feature weighting significantly changes which customers are prioritized as high-risk, even when the underlying churn signal remains unchanged.

Figure 17 reports Precision@K for churn detection. For both models, the unweighted baseline achieves highest precision across all K values. Lecture-weighted configuration yields comparable but slightly reduced precision, while demographic and financial weightings perform markedly worse. This demonstrates that stronger feature focusing does not improve churn detection and may degrade top-K prediction quality.

Overall, feature weighting substantially affects similarity distributions and rankings. While certain weight vectors increase semantic contrast, they do not improve downstream task performance, highlighting a trade-off between discriminatory span and predictive reliability.

6. Discussion

The central focus of this work was, on the one hand, to present and verify Bordawekar and Shmueli’s approach to extending RDBMS with semantic queries and, on the other hand, to optimize this approach for the database used. The findings obtained in the process are discussed below.

6.1 Summary of Key Findings

The results address two main aspects: the properties of the learned vector spaces and the behavior of similarity operators. Using a composite quality metric, the choice of Word2Vec algorithm proved to be the dominant factor influencing vector quality. CBOW produced more consistent and structured semantic spaces for the relational dataset, whereas SGNS performed worse in both computational and visual analyses. The remaining parameters showed a rapid quality improvement from minimal to moderate values, followed by a clear plateau in which further increases produced no measurable benefit. Primary key embeddings encoded no reliable row-level semantic information, as key vectors showed no systematic proximity to values originating from their corresponding rows.

Similarity operators strongly influenced the structure of neighborhoods, particularly among the most similar entities. Weighting further altered these neighborhoods, yet neither operator selection nor weighting improved churn detection performance. The experiments therefore indicate that similarity structure and task relevance are only loosely connected.

6.2 Discussion of Embedding Quality

Although Bordawekar and Shmueli do not explicitly address model quality, it substantially affects the semantics encoded in the vectors. Parameter selection must therefore be adapted to the dataset characteristics when implementing Semantic SQL. The following sections interpret the observed parameter effects for the database used.

6.2.1 CBOW vs. SGNS DIVERGENCE

The clearest observation is that CBOW generates substantially more separable cluster structures than SGNS, whose vectors overlap strongly across categories. At first glance, this contradicts prior work reporting superior semantic performance of SGNS Mikolov, Sutskever, et al. (2013). However, the discrepancy is explained by corpus size and data structure. Previous studies train SGNS on corpora containing billions of tokens and explicitly emphasize the necessity of large datasets. Even reductions within the billion-token range measurably degrade analogy performance, and similar effects are reported for million-token corpora Sahlgren and Lenci (2016).

The database used here contains only about 100,000 semantically relevant tokens and therefore provides only a sparse co-occurrence signal. Under these conditions, stable semantic relations cannot be estimated reliably. The apparent superiority of CBOW is therefore not higher semantic accuracy but a consequence of its learning objective. CBOW predicts targets from averaged context vectors (Mikolov, Chen, et al., 2013), which smooths random observations and stabilizes representations in small corpora. SGNS instead optimizes individual word-context pairs (Mikolov, Chen, et al., 2013) and depends on diverse contexts to obtain reliable estimates. With insufficient diversity, the model learns local associations rather than global relations, producing fragmented and overlapping clusters.

The observed CBOW advantage is consequently a regularization effect. SGNS exhibits its known strengths only with sufficiently large and context-rich data, whereas in small relational corpora smoother objectives are inherently more robust.

6.2.2 PARAMETER SENSITIVITY AND OPTIMAL RANGES

The OAT and grid search findings show that the parameters have the greatest influence when transitioning from minimum to moderate values. After that, a plateau is reached where medium and high parameter values have no further measurable influence on the model quality. This is probably also a consequence of the small and homogeneous data set. Thus, even a small number of training epochs is sufficient to encode the existing semantic structures in the vector model, and further training runs merely confirm the structure already learned without adding new information. The same applies to sentence repetitions: even a high repetition rate does not generate new semantic relations, as the data set does not contain them. The same principle applies to the vector dimension, since additional dimensions cannot encode semantic structures that are not present in the data set. The optimal parameters for this database are therefore at the beginning of the plateau, where the model quality has already stabilized, but the training duration is not unnecessarily extended by the parameters.

6.2.3 FAILURE OF PRIMARY KEY EMBEDDINGS

The finding that primary key embeddings are not located close to their associated row vectors was, to some extent, anticipated. Word2Vec algorithms group tokens that appear in similar contexts together in the vector space. Since primary keys occupy structurally similar positions across rows – independent of the semantic content of their corresponding entries – they tend to share contextual patterns with other keys rather than with the values of their own row. Consequently, key vectors cluster near other key vectors rather than near

their associated row values. This does not allow any conclusions to be drawn about the ability of the key vectors to encode semantic information in their lines.

6.3 Semantic Similarity Operators

The results show that different similarity operators result in significantly different similarity neighborhoods.

6.3.1 OPERATOR CHARACTERISTICS AND SELECTION

The various operators actually capture different forms of similarity. While ProximityAvg and ProximityAvgAll target the global similarity of entire database rows, ProximityTopNAvg and SubsetProximityAvg focus on the most similar sub-features, depending on the parameters topN and k, respectively, and ProximityMax merely identifies the best-matching individual feature between two entities.

ProximityAvg achieves the clearest separation between similar and dissimilar customers, while ProximityAvgAll predominantly delivers values close to zero and allows for little differentiation. Comparing centroids is therefore superior to equally combining all feature combinations, as the latter dilutes the comparison due to widely varying attributes from different rows. To fix this problem, the operator from ProximityAvgAll must be adjusted so that it only compares attributes that come from the same row, similar to the operator with weighted attributes.

ProximityTopNAvg and SubsetProximityAvg also yield meaningful neighbourhoods, but these are significantly different from those obtained with ProximityAvg. Especially with smaller values of topN or k, the similarity shifts from a global profile similarity to a structural partial similarity. Entities are then no longer considered similar overall, but similar in certain characteristic aspects, resulting in several simultaneously valid similarity relationships within the same database.

ProximityMax shows lower selectivity, since individual attributes that match strongly lead to high similarity values even for customers who are otherwise different. The operator therefore measures the presence of common characteristics rather than actual overall similarity, making it less suitable as a ranking operator and more suitable as a semantic match operator.

Overall, the results illustrate that similarity in relational embeddings is not a clear-cut value. Rather, the choice of operator determines whether a query searches for similar overall profiles, structurally comparable subgroups, or merely common characteristics. Semantic SQL therefore not only formulates a query to the data, but also a decision about what meaning “similar” should have in this context.

6.3.2 FEATURE WEIGHTING: A DOUBLE-EDGED SWORD

The weighting of attributes further alters the similarity relation. While the unweighted operators reflect the semantic structure learned through training, weighting shifts the focus specifically to certain aspects of the entities, resulting in significantly different neighbourhoods even though the underlying vectors do not change.

It is particularly striking that the most similar customers change significantly, while the most dissimilar remain largely stable. This suggests that large differences between entities

are clearly recognisable, while similarity itself remains subject to interpretation, meaning weighting acts less like a fine adjustment and more like a choice of perspective on the data.

At the same time, greater differentiation of similarity values does not automatically lead to better results in the application. Although weighting leads to more clearly separated score distributions, the identification of customers at risk of churn does not improve and in some cases even deteriorates. It follows that a semantically more plausible similarity does not necessarily correspond to the similarity relevant to the task.

6.3.3 APPLICATION TO CHURN DETECTION

The use of similarity scores for churn detection reveals an important limitation of semantic queries. Similarity to customers who have already cancelled their contracts can serve as an indication, but it does not constitute a prediction in the statistical sense, since the embeddings map existing structures in the data set but do not optimise them for the target feature.

The fact that the unweighted model achieves the best results suggests that the overall structure created by the training is closer to the actual churn relevance than specifically highlighted individual aspects. Although weighting increases interpretability, it removes the score from the actual decision-making goal.

This reveals a fundamental difference between semantic similarity and predictive relevance. Semantic SQL can reveal relationships and support exploratory analyses, but it does not replace a prediction model optimised for target variables.

6.4 Limitations

While the findings provide insight into the behavior of semantic SQL on relational data, they are subject to several important limitations.

6.4.1 DATASET-SPECIFIC CONSTRAINTS

A central limitation of this work is the size and structure of the database. As discussed in Section 6.2.1, the generated corpus is several orders of magnitude smaller than corpora typically used to train Word2Vec models in NLP. Consequently, the learned vectors rely on sparse co-occurrence information and are therefore less statistically stable. This limitation does not invalidate the applicability of Semantic SQL itself, but it restricts the generalizability of the results. The dataset mainly consists of structured attribute values with limited contextual diversity and, compared to natural language text, fewer independent semantic relations can be observed. As a result, the embedding space primarily reflects attribute distributions rather than rich conceptual relations.

Furthermore, several extensions proposed by Bordawekar and Shmueli (2017), such as the integration of external corpora, pre-trained embeddings, or multimodal information, were not considered. The conclusions of this work therefore apply primarily to isolated relational datasets.

6.4.2 METHODOLOGICAL LIMITATIONS

The quality metrics used are plausible surrogates that evaluate the geometric structure of the vector space and the grouping of sentence vectors that are expected to match. While these are valid indicators of semantic encoding, they do not represent a semantic “ground truth” or precisely measure the quality of encoded semantics. Similarly, the stability analysis of cosine similarities across different seeds is valuable for practical model application but does not directly assess semantic encoding quality. Similarly, t-SNE is a practical tool for visualizing vectors, but the resulting cluster optics should not be overinterpreted.

6.4.3 SCOPE OF THE EXPERIMENTS

The experimental evaluation focuses primarily on similarity-based semantic operators. Other aspects of Semantic SQL described by Bordawekar and Shmueli (2017), such as analogy, clustering, or prediction queries, were not systematically investigated.

Additionally, the experiments examining whether key vectors are positioned close to the attribute values originating from their corresponding database rows are based on a strong interpretation of row representation, namely that a key embedding not only encodes the semantic content of its tuple but is located in a similar position in the vector space. However, the concept of Semantic SQL does not necessarily require such a direct correspondence. Instead, key vectors are intended to represent tuples relationally: tuples with similar attribute structures should appear close to each other in the key vector space, meaning the semantic meaning of a key emerges through its neighborhood among other keys rather than through proximity to its own attribute values.

This relational key-to-key similarity was not explicitly evaluated in this work. The negative findings therefore only rule out a direct attribute-encoding interpretation of key vectors, but not their potential use as relational tuple representations. The work therefore validates only a subset of the Semantic SQL concept, namely similarity-based relational reasoning.

6.5 Comparison with Literature

In this work, it was possible to verify various aspects of the concept proposed by Bordawekar and Shmueli (2017, 2019).

6.5.1 ALIGNMENT WITH BORDAWEKAR AND SHMUELI

The experiments support the central idea of Semantic SQL proposed by Bordawekar and Shmueli (2017), namely that semantic relations can be derived from relational data by transforming database tuples into a textual representation and applying embedding models. The learned vector spaces consistently encode meaningful relations between attribute values and allow similarity-based queries to retrieve plausible neighboring tuples.

In particular, the results demonstrate that multiple valid similarity relations can exist simultaneously within the same dataset. Depending on the chosen operator, tuples can be considered similar either in their overall profile or in specific structural aspects. This behavior aligns with the intention of Semantic SQL to extend classical database queries by semantic retrieval rather than exact matching.

6.5.2 DEVIATIONS AND NEW INSIGHTS

While the general feasibility of Semantic SQL is supported, the experiments also reveal several practical differences from the conceptual presentation in the literature. First, the choice of embedding model is of greater importance than expected. In contrast to typical NLP applications, CBOW produces more stable and interpretable vector spaces than SGNS for the structured relational dataset, suggesting that relational corpora behave differently from natural language corpora and require different considerations regarding parameter choices.

Second, the experiments show that similarity itself is not a single well-defined concept. Different operators capture global similarity, partial similarity, or isolated attribute matches, leading to substantially different neighborhoods, and consequently Semantic SQL queries implicitly define the semantics of similarity through operator selection.

Finally, increasing semantic interpretability through feature weighting does not necessarily improve task performance. The churn experiments illustrate that semantically plausible similarity can diverge from predictive relevance, highlighting the distinction between exploratory semantic querying and supervised prediction tasks.

7. Conclusion

This work aimed to extend the work of Bordawekar and Shmueli (2017, 2019) to include insights into small, highly structured databases. It was shown that the transfer of the database into vector space is possible using the proposed Word2Vec algorithms and delivers stable, high-quality results across a wide range of training parameters. Furthermore, the applicability of two central concepts of the proposed CI queries — semantic similarity and semantic clustering — could be verified using this vector model.

Addressing RQ1, only a limited influence of the model parameters on the quality of the vector model could be demonstrated in extensive test series. The vector quality was only negatively affected for very small parameter values and was consistently good overall, with far better results for the CBOW algorithm compared to SGNS. Regarding RQ2, the intuitive assumption that a key vector is located in the vector space close to the vectors of its corresponding row values was refuted. This finding clarifies the nature of key representations and motivates an alternative interpretation of key vectors as relational tuple descriptors, whose semantic meaning emerges through proximity to other keys rather than to their own attribute values.

Building on these vector representations, the experiments addressing RQ3 and RQ4 show that different implementations of similarity comparisons are necessary in order to allow precise and targeted comparisons, as each operator captures a distinct notion of similarity. However, while feature weighting further alters the resulting similarity rankings and increases discriminatory span, this does not translate into improved downstream task performance. Addressing RQ5, clustering as described by Bordawekar and Shmueli allows database rows to be grouped according to selected characteristics, enabling the identification of customers likely to churn, as long as a collection of representative rows with these characteristics is available for prototype construction.

However, these experimentally obtained findings only cover a small part of the concept of a semantic database proposed by Bordawekar and Shmueli and cannot fully verify its

applicability to the selected database. Furthermore, the information obtained with this specific dataset can only be transferred to other databases to a limited extent.

Future work should aim to make a direct comparison between large, comprehensive databases and smaller ones in order to clarify the differences, particularly with regard to the poor performance of the SGNS algorithm. In addition to similarity and clustering, the authors suggest other functions that should also be verified on small datasets in further work, including analogies and the integration of external text corpora. Finally, there is a lack of implementation of a complete pipeline from the textification of a database, through the implementation of semantic functions as UDFs, to the creation of complete CI queries that allow semantic queries directly from the database.

8. Bibliography

References

- Bordawekar, R., & Shmueli, O. (2016). Enabling cognitive intelligence queries in relational databases using low-dimensional word embeddings. *CoRR*, *abs/1603.07185*. <http://arxiv.org/abs/1603.07185>
- Bordawekar, R., & Shmueli, O. (2017). Using word embedding to enable semantic queries in relational databases. *Proceedings of the 1st Workshop on Data Management for End-to-End Machine Learning*. <https://doi.org/10.1145/3076246.3076251>
- Bordawekar, R., & Shmueli, O. (2019). Exploiting latent information in relational databases via word embedding and application to degrees of disclosure. *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. <http://cidrdb.org/cidr2019/papers/p27-bordawekar-cidr19.pdf>
- Cafarella, M., Halevy, A., Wang, D., Wu, E., & Zhang, Y. (2008). Webtables: Exploring the power of tables on the web. *PVLDB*, *1*, 538–549. <https://doi.org/10.14778/1453856.1453916>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Commun. ACM*, *13*(6), 377–387. <https://doi.org/10.1145/362384.362685>
- DB-Engines Ranking*. (2025). DB-Engines. Retrieved December 18, 2025, from <https://db-engines.com/en/ranking>
- Ferrone, L., & Zanzotto, F. M. (2020). Symbolic, distributed, and distributional representations for natural language processing in the era of deep learning: A survey. *Frontiers in Robotics and AI, Volume 6 - 2019*. <https://doi.org/10.3389/frobt.2019.00153>
- Firth, J. R. (1957). A synopsis of linguistic theory, 1930–1955 [Special volume of the Philological Society]. In *Studies in linguistic analysis* (pp. 1–32). Blackwell.
- Gray, J., & Reuter, A. (1993). *Transaction processing: Concepts and techniques*. Morgan Kaufmann.
- Harris, Z. S. (1954). Distributional structure. <https://api.semanticscholar.org/CorpusID:86680084>
- Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition, with language models* (3rd) [Online manuscript released January 6, 2026]. <https://web.stanford.edu/~jurafsky/slp3/>
- Landauer, T. K., & Dumais, S. T. (1997). A solution to Plato’s problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge. *Psychological Review*, *104*(2), 211–240. <https://doi.org/10.1037/0033-295X.104.2.211>
- Levy, O., & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, & K. Weinberger (Eds.), *Advances in neural information processing systems* (Vol. 27). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2014/file/b78666971ceae55a8e87efb7cbfd9ad4-Paper.pdf
- Matplotlib Development Team. (2025). Matplotlib: Visualization with python [Accessed January 22, 2026].

- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *Proceedings of Workshop at ICLR, 2013*.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In C. Burges, L. Bottou, M. Welling, Z. Ghahramani, & K. Weinberger (Eds.), *Advances in neural information processing systems* (Vol. 26). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf
- Project Jupyter. (2025). Project jupyter [Zugriff am 22.01.2026].
- Řehůřek, Radim et al. (2025). Word2vec — gensim documentation [Zugriff am 22.01.2026].
- Relational database management system market report*. (2024). Market Reports World. Retrieved December 26, 2025, from <https://www.marketreportsworld.com/market-reports/relational-database-management-system-market-14721038>
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. [https://doi.org/https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/https://doi.org/10.1016/0377-0427(87)90125-7)
- Sahlgren, M., & Lenci, A. (2016, November). The effects of data size and frequency range on distributional semantic models. In J. Su, K. Duh, & X. Carreras (Eds.), *Proceedings of the 2016 conference on empirical methods in natural language processing* (pp. 975–980). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D16-1099>
- scikit-learn Developers. (2025). Scikit-learn: Machine learning in python [Accessed January 22, 2026].
- Seaborn Developers. (2025). Seaborn: Statistical data visualization [Accessed January 22, 2026].
- Topre, G. (2022). Bank customer churn dataset [Accessed: 2026-01-18].
- Turney, P. (2006). Similarity of semantic relations. *Computational Linguistics*, 32, 379–416. <https://doi.org/10.1162/coli.2006.32.3.379>
- Venetis, P., Halevy, A., Madhavan, J., Paşca, M., Shen, W., Wu, F., Miao, G., & Wu, C. (2011). Recovering semantics of tables on the web. *Proc. VLDB Endow.*, 4(9), 528–538. <https://doi.org/10.14778/2002938.2002939>

Appendix A. Tables

Table 6: Word2Vec default hyperparameters for parameters not varied in experiments

Parameter	Default Parameter	Description of Effects
Vector Size	150	Embedding dimensionality. Higher values capture more detail but increase computational cost and overfitting risk.
Window Size	4	Defines the context window. Smaller values focus on syntactic relations, larger ones on semantic/thematic relations.
Min Count	1	Minimum word frequency. Higher values reduce vocabulary size and noise but may remove rare tokens.
Epochs	30	Passes over the corpus. More epochs improve convergence but can cause overfitting.
Learning Rate (Alpha)	0.025	Step size for gradient updates. High values cause instability; low values slow training. Often decays.
Model Type	SGNS und CBOW	Skip-gram: better for semantics/rare words. CBOW: faster, better for syntax/frequent words.
Negative Sampling	5	Number of negative samples. Balances efficiency and accuracy. Higher values improve quality.

Appendix B. Formal Definitions of Proximity Operators

Let $S_A = \{v_1^A, \dots, v_{n_A}^A\}$ and $S_B = \{v_1^B, \dots, v_{n_B}^B\}$ be two sets of embedding vectors, with cardinalities $n_A = |S_A|$ and $n_B = |S_B|$.

Cosine similarity between two vectors x and y is defined as:

$$S_c(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}. \quad (1)$$

B.1 ProximityMax

PROXIMITYMAX is defined as the maximum cosine similarity over all pairs of vectors from S_A and S_B :

$$\text{PROXIMITYMAX}(S_A, S_B) = \max_{\substack{1 \leq i \leq n_A \\ 1 \leq j \leq n_B}} S_c(v_i^A, v_j^B). \quad (2)$$

B.2 ProximityAvg

PROXIMITYAVG computes cosine similarity between the centroid vectors of S_A and S_B . The centroids are defined as:

$$v_{\text{avg}}^A = \frac{1}{n_A} \sum_{i=1}^{n_A} v_i^A, \quad (3)$$

$$v_{\text{avg}}^B = \frac{1}{n_B} \sum_{j=1}^{n_B} v_j^B. \quad (4)$$

The operator is then given by:

$$\text{PROXIMITYAVG}(S_A, S_B) = S_c(v_{\text{avg}}^A, v_{\text{avg}}^B). \quad (5)$$

B.3 ProximityTopNAvg

Let $\{S_c(v_i^A, v_j^B)\}_{i,j}$ denote the set of all pairwise cosine similarities between vectors in S_A and S_B . These values are sorted in non-increasing order:

$$s_{(1)} \geq s_{(2)} \geq \dots \geq s_{(n_A n_B)}.$$

Let $N' = \min(N, n_A n_B)$. Then PROXIMITYTOPNAVG is defined as:

$$\text{PROXIMITYTOPNAVG}(S_A, S_B) = \frac{1}{N'} \sum_{t=1}^{N'} s_{(t)}. \quad (6)$$

The constraint $N < n_A n_B$ is assumed.

B.4 ProximityAvgAll

PROXIMITYAVGALL computes the mean of all pairwise cosine similarities:

$$\text{PROXIMITYAVGALL}(S_A, S_B) = \frac{1}{n_A n_B} \sum_{i=1}^{n_A} \sum_{j=1}^{n_B} S_c(v_i^A, v_j^B). \quad (7)$$

B.5 SubsetProximityAvg

For any subset $T \subseteq S$ with $|T| = k$, define its centroid as:

$$\text{centroid}(T) = \frac{1}{k} \sum_{v \in T} v. \quad (8)$$

Let S_A^k (respectively S_B^k) denote the set of centroid vectors of all k -element subsets of S_A (respectively S_B). Then SUBSETPROXIMITYAVG is defined as:

$$\text{SUBSETPROXIMITYAVG}(k, S_A, S_B) = \max_{\substack{a \in S_A^k \\ b \in S_B^k}} S_c(a, b). \quad (9)$$

Appendix C. Metric Definitions

C.1 Silhouette Coefficient

$$SC_{\text{global}} = \frac{1}{|R|} \sum_{r \in R} \max \left(0, \frac{b(r) - a(r)}{\max\{a(r), b(r)\}} \right) \quad (10)$$

C.2 Rank Stability

$$RS_{\text{global}} = \frac{1}{|R|} \sum_{r \in R} \left(\frac{1}{|P|} \sum_{(i,j) \in P} \frac{|N_k^{(i)}(r) \cap N_k^{(j)}(r)|}{|N_k^{(i)}(r) \cup N_k^{(j)}(r)|} \right) \quad (11)$$

C.3 Feature Coherence

$$FC_{\text{knn}} = \frac{1}{|R|} \sum_{r \in R} \left(\frac{1}{k} \sum_{n \in N_k(r)} \frac{|F(r) \cap F(n)|}{|F(r) \cup F(n)|} \right) \quad (12)$$

$$FC_{\text{norm}} = \frac{FC_{\text{knn}} - FC_{\text{random}}}{1 - FC_{\text{random}}} \quad (13)$$

C.4 Semantic SQL Model Quality

$$SSMQ = \alpha \cdot RS + \beta \cdot SC + \gamma \cdot FC \quad (14)$$

C.5 Key Embedding Metrics

$$APS = \frac{\text{Number of found row values}}{\text{Number of row values in vocabulary}} \quad (15)$$

$$WAPS = \frac{\sum_{i \in \text{found}} \frac{1}{r_i + 1}}{\text{Number of row values in vocabulary}} \quad (16)$$

C.6 Weighted Similarity

$$S_{c,w}(\text{row}_a, \text{row}_b) = \frac{\sum_{i=1}^W w_i S_c(\text{row}_{a,i}, \text{row}_{b,i})}{\sum_{i=1}^W w_i} \quad (17)$$

Appendix D. Additional Data

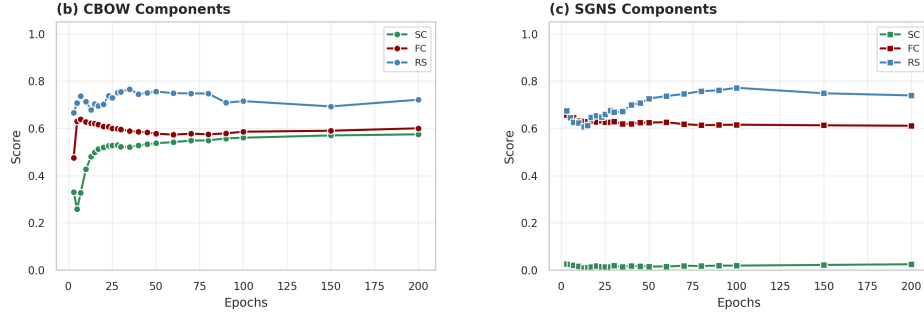


Figure 18: Impact of training epochs on model quality metrics. Individual metric components (SC, FC, RS) for CBOW (left) and SGNS (right).

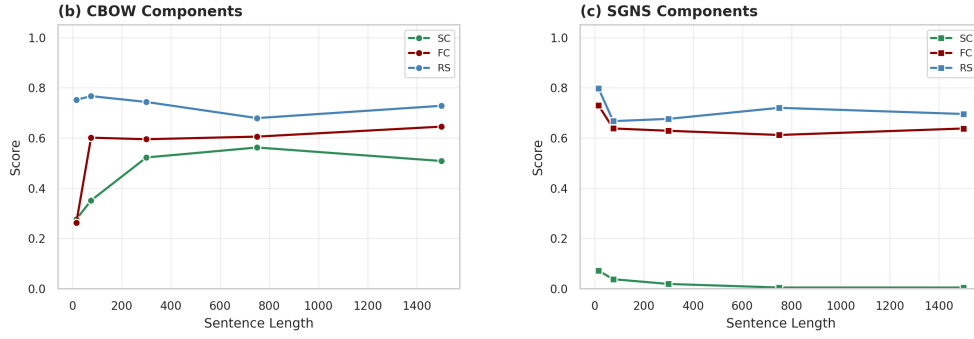


Figure 19: Impact of sentence length (word repetitions) on model quality metrics. Layout as in Figure 18.

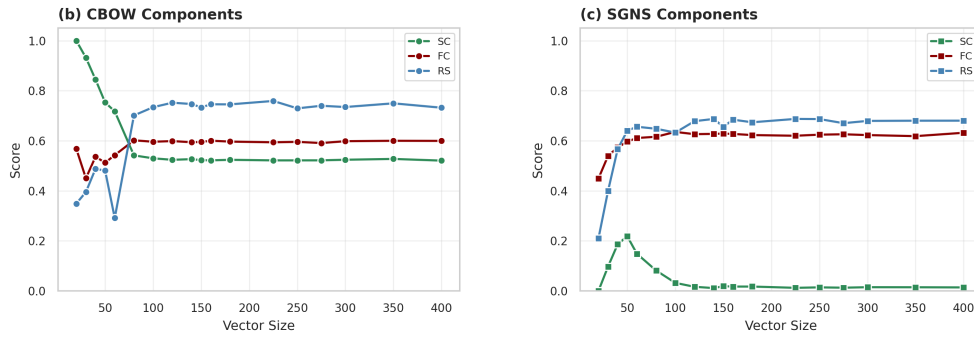


Figure 20: Impact of embedding vector size on model quality metrics. Layout as in Figure 18.

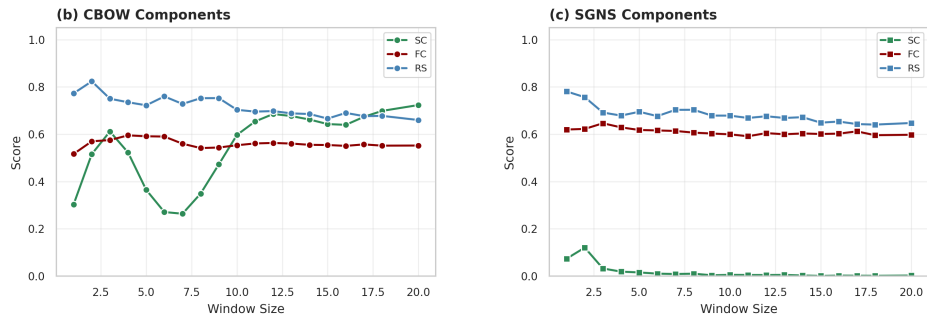


Figure 21: Impact of context window size on model quality metrics. Layout as in Figure 18.